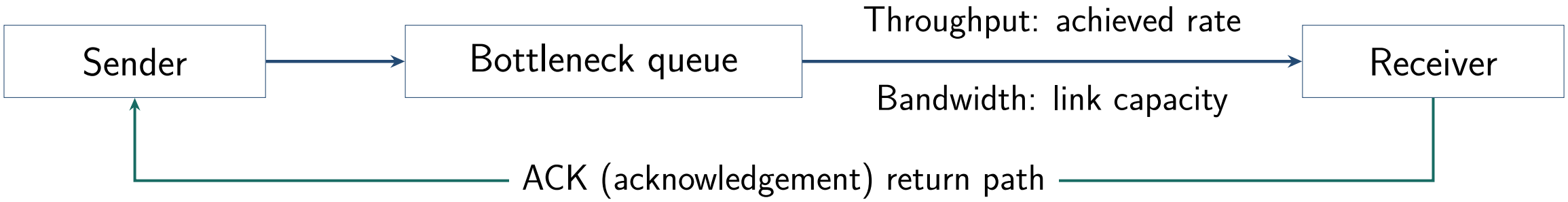
AdvNet

Revealing Performance Issues in Network Protocols
by Generating Adversarial Environments

Presenters: Eric Gao and Xiaoyang Chen

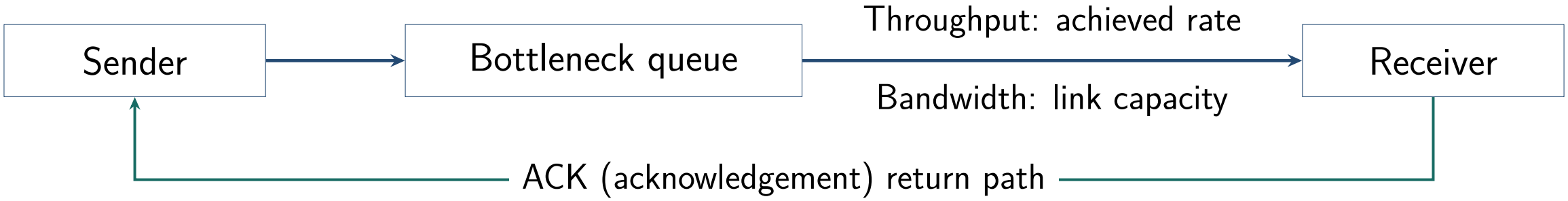
Background: Congestion Control



Unknown available bandwidth

- Send too slowly $\rightarrow$ underutilize bandwidth
- Send too fast $\rightarrow$ queue buildup, higher round-trip time (RTT), packet loss

Background: Congestion Control



Unknown available bandwidth

- Send too slowly → underutilize bandwidth
- Send too fast → queue buildup, higher round-trip time (RTT), packet loss

Feedback-driven adaptation

- Signals: ACKs, delay, packet loss
- Controls: congestion window (in-flight limit) and pacing (send timing)
- Loss recovery: duplicate ACKs and retransmission timeouts (RTO)

Motivation: Congestion Control Is Hard to Test

Strict goals in dynamic networks

- High throughput, low delay, and low loss under changing links
- Fast convergence to stable, fair rates when flows compete

Adapted from PCC Vivace: Online-Learning Congestion Control §1

Motivation: Congestion Control Is Hard to Test

Strict goals in dynamic networks

- High throughput, low delay, and low loss under changing links
- Fast convergence to stable, fair rates when flows compete

Adapted from PCC Vivace: Online-Learning Congestion Control §1

Consequences for testing

- A large parameter space makes thorough testing difficult.
- Protocol updates and new designs may pass standard tests yet hide severe regressions.
- Goal: automatically find environments that expose performance problems.

Motivation: Existing Testing Approaches Fall Short

Hand-designed tests

- Manually chosen bandwidth and latency ranges: easy to interpret and useful for checking suspected issues
- Coverage depends on the designer; protocol state makes the sequence space grow exponentially with sequence length.

Motivation: Existing Testing Approaches Fall Short

Hand-designed tests

- Manually chosen bandwidth and latency ranges: easy to interpret and useful for checking suspected issues
- Coverage depends on the designer; protocol state makes the sequence space grow exponentially with sequence length.

Real-world traces and testbeds

- Coverage is bounded by collection sites and times.
- No active search for worst cases; live anomalies are hard to reproduce.

Mahimahi	Record-and-replay link emulator
Pantheon	Congestion-control benchmark across real and emulated paths
Puffer	Live adaptive video streaming testbed

Adversarial Testing: Simulation vs. Emulation

Simulation (ns-3)	Simplified models: fast and deterministic, but behavior can differ from deployed code.
Emulation (Mahimahi)	Real OS kernel stack: exposes implementation bugs, but execution is slow and noisy.

Adversarial Testing: Simulation vs. Emulation

Simulation (ns-3)	Simplified models: fast and deterministic, but behavior can differ from deployed code.
Emulation (Mahimahi)	Real OS kernel stack: exposes implementation bugs, but execution is slow and noisy.

Prior work: CC-Fuzz (HotNets '22)

- Uses a genetic algorithm (GA) in ns-3 simulation.
- Represents links as low-level Packet Delivery Opportunities (PDOs).
- Struggles to find strong adversarial examples within a limited emulation budget

Will explain later

AdvNet: Inputs and Reference

Inputs

- Parameter bounds
- Target protocol
- Reference baseline

Reference options

- Another protocol
- An older version
- A single-path configuration
- A capacity benchmark

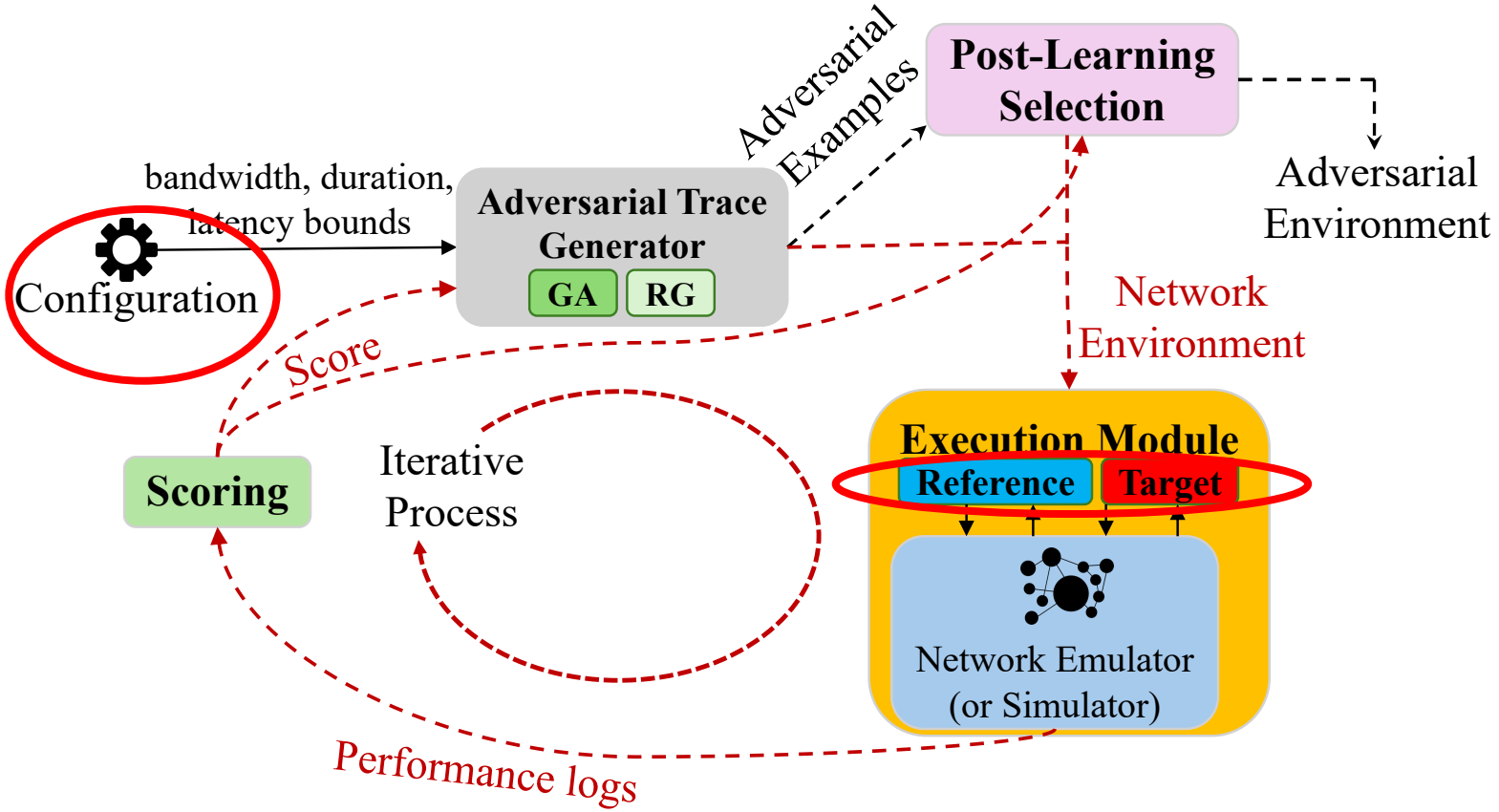


Fig. 1, AdvNet.

AdvNet: Execution Loop

1. Generate a candidate trace.
2. Run target and reference in the same environment.
3. Score their performance gap and feed it back to the generator.

Mahimahi is extended to support time-varying delay and dual paths.

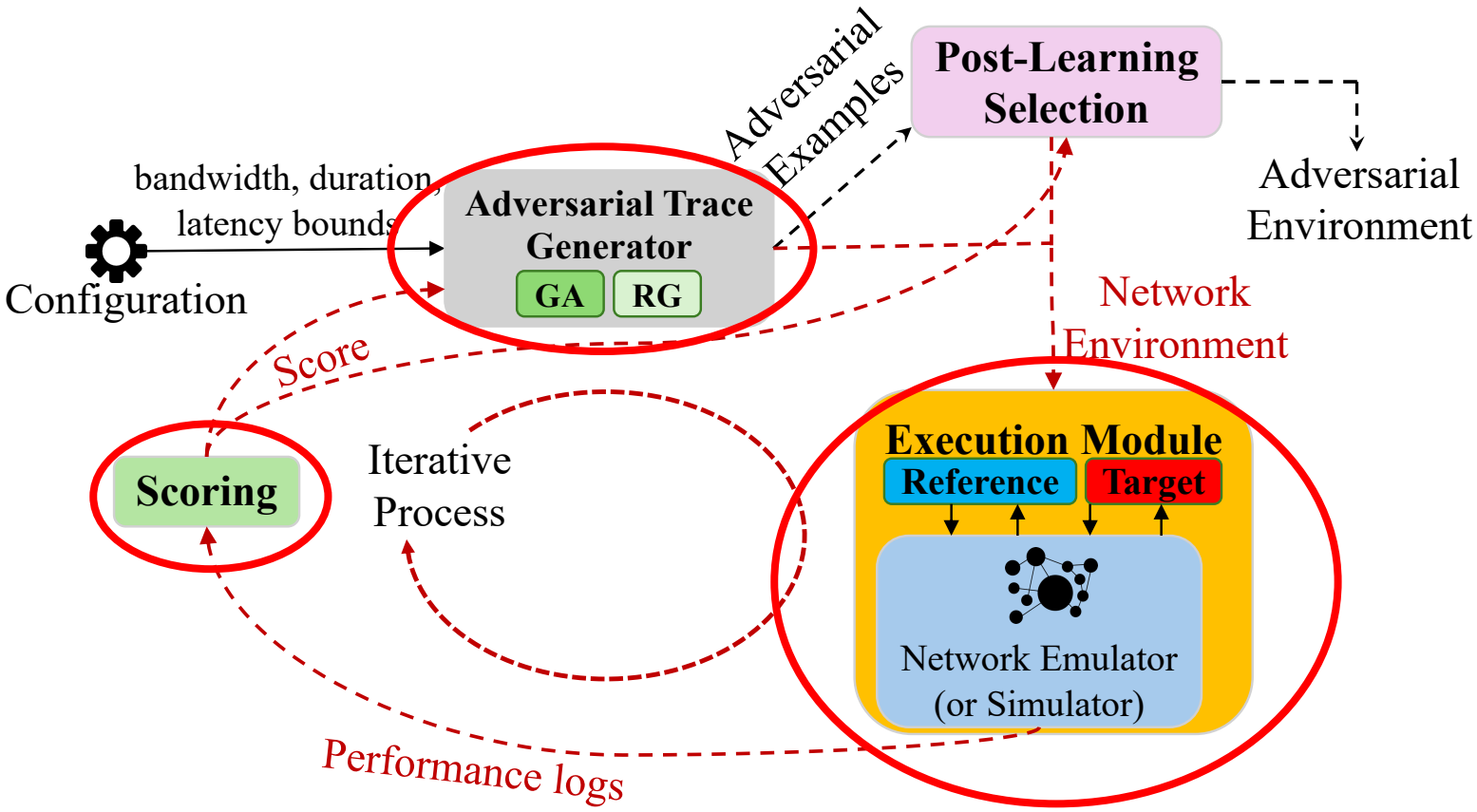


Fig. 1, AdvNet.

AdvNet: Output Stage

Post-Learning Selection

1. Reserve part of the budget to re-test candidates.
2. Select the trace with the most reliably high average score.

After selection: independently re-evaluate the selected trace.

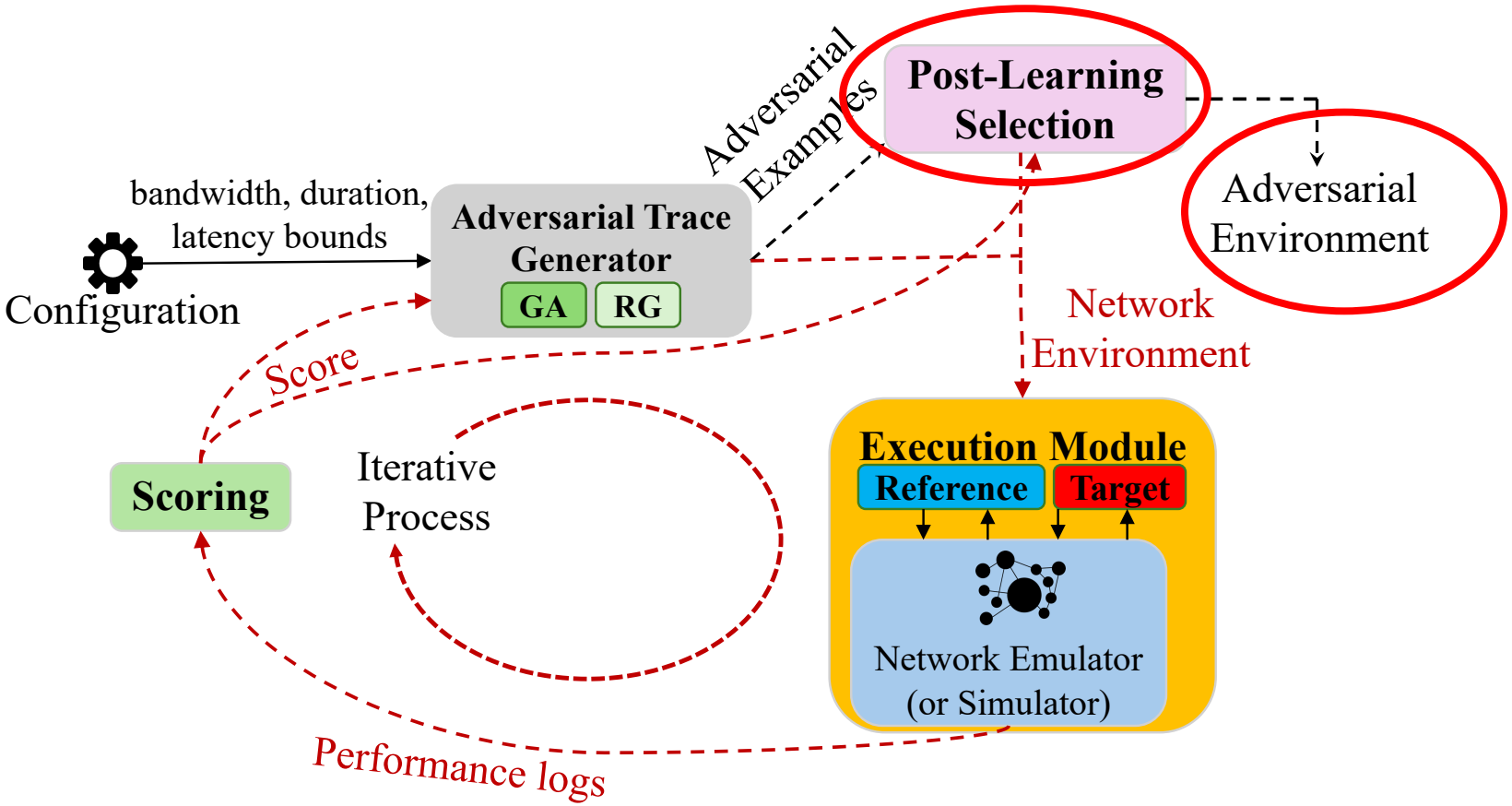


Fig. 1, AdvNet.

Scoring the Performance Gap

R: reference performance T: target performance

$$\text{score} = \frac{R - T}{\max(R, T)}$$

Higher metric values are better (e.g., throughput).

Scoring the Performance Gap

R: reference performance T: target performance

$$\text{score} = \frac{R - T}{\max(R, T)}$$

Higher metric values are better (e.g., throughput).

Reference: 10 Mbps Target: 2 Mbps

$$\text{score} = \frac{10 - 2}{10} = 0.8$$

- Lower is better (e.g., completion time): use $T - R$ in the numerator.
- Score lies in $[-1, 1]$; positive scores mean the target underperforms.

Combining Throughput and Delay

$$U = t_{\text{coeff}} \frac{\tau}{\tau_{\text{max}}} + (1 - t_{\text{coeff}}) \frac{d_{\text{min}}}{d}$$

τ: throughput d: average packet delay

τ_{max}: maximum-throughput benchmark

d_{min}: minimum-delay benchmark

Combining Throughput and Delay

$$U = t_{\text{coeff}} \frac{\tau}{\tau_{\max}} + (1 - t_{\text{coeff}}) \frac{d_{\min}}{d}$$

τ: throughput d: average packet delay

$\tau_{\max}$: maximum-throughput benchmark
$d_{\min}$: minimum-delay benchmark

Throughput weight $t_{\text{coeff}} \in [0, 1]$:

1: throughput only	0: delay only	0.5: equal weight

Compute reference and target utilities using the same benchmarks and weight, then use them as R and T in the performance-gap score.

Two Core Design Challenges

Sample efficiency

- Real protocol executions take seconds per evaluation.
- A large trace space and emulation noise limit the search.
- Repeating every trace leaves less budget for exploring new ones.

Two Core Design Challenges

Sample efficiency

- Real protocol executions take seconds per evaluation.
- A large trace space and emulation noise limit the search.
- Repeating every trace leaves less budget for exploring new ones.

Practical usefulness

- Zero bandwidth makes every protocol get zero throughput.
- Sudden bandwidth increases naturally cause brief underutilization.
- Meaningful reference comparisons must match the developer's task.

Network Environment Representation

$$T = (B, L, D, I)$$

B: bandwidth sequence

D: interval durations

L: one-way delay sequence

I: fixed settings, such as queue length

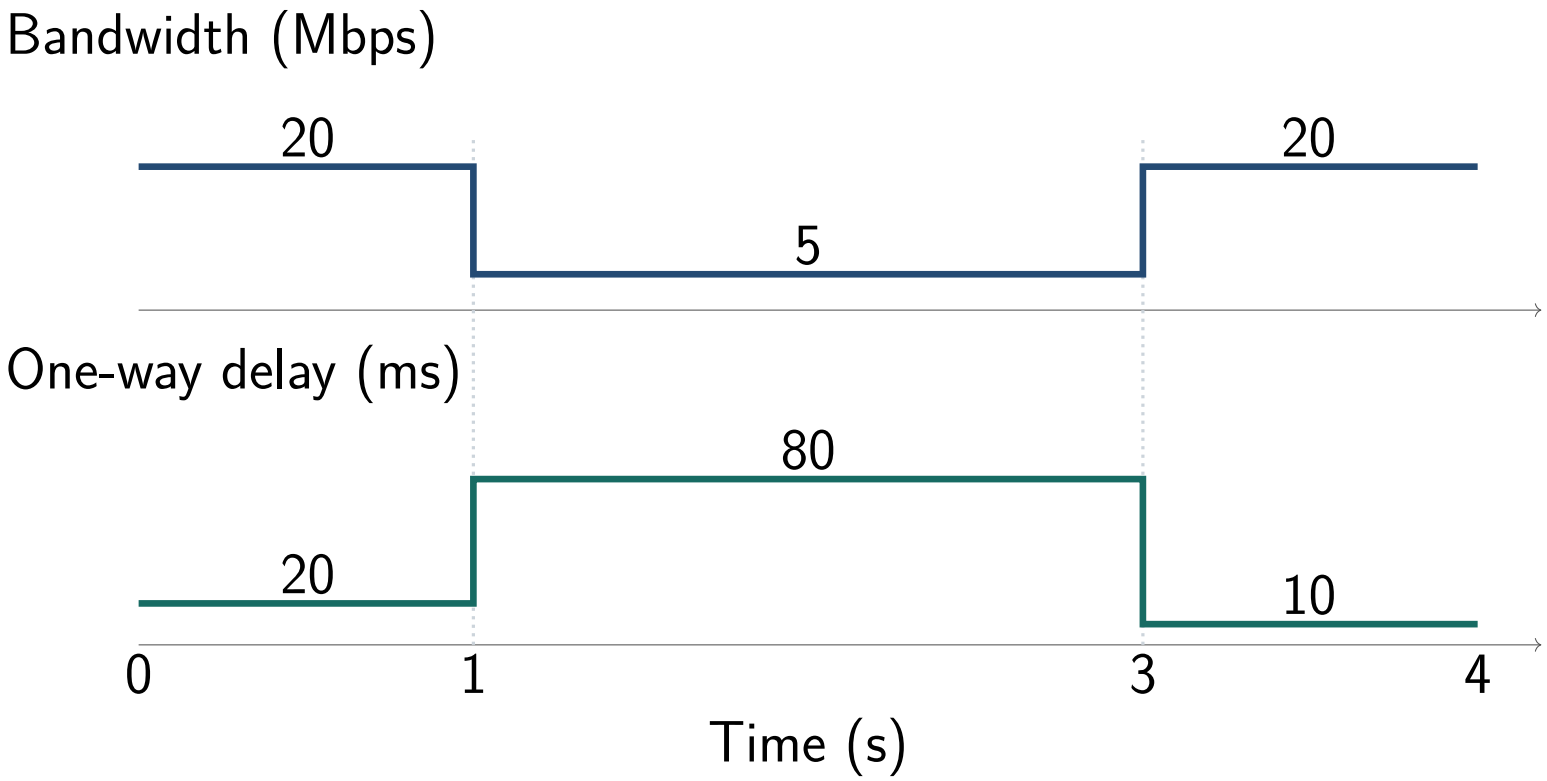
Multipath: separate B, L for each link; shared interval durations D.

Each parameter is constrained by user-specified bounds.

	Bandwidth	**Latency**	**Duration**	**Queue Length**
Lower Bound	1 Mbps	5 ms	0.5 s	500
Upper Bound	100 Mbps	100 ms	2.5 s	10000

Table 1, AdvNet.

An Example Network Trace



$B = (20, 5, 20)$ Mbps

$L = (20, 80, 10)$ ms

$D = (1, 2, 1)$ seconds

In the middle interval, bandwidth drops while delay rises.

All values satisfy the bounds on the previous slide.

Interval Representation vs. PDOs

PDO: a packet delivery opportunity specifies when a packet may leave.

TCP Protocol	AdvNet	CC-Fuzz
bbr	79.11%	14.31%
cubic	46.24%	29.78%
vegas	84.71%	27.29%
reno	41.26%	28.8%

- AdvNet uses interval representation; CC-Fuzz uses PDO representation.
- Score: throughput shortfall relative to capacity.
- A higher score means a more adversarial environment was found.

Table 2, AdvNet.

Interval Representation vs. PDOs

PDO: a packet delivery opportunity specifies when a packet may leave.

TCP Protocol	AdvNet	CC-Fuzz
bbr	79.11%	14.31%
cubic	46.24%	29.78%
vegas	84.71%	27.29%
reno	41.26%	28.8%

- AdvNet uses interval representation; CC-Fuzz uses PDO representation.
- Score: throughput shortfall relative to capacity.
- A higher score means a more adversarial environment was found.

Why intervals help

- Lower dimensionality: increasing 25 $\rightarrow$ 100 PDOs drops BBR's score from 14.31% to 5.91%.
- Explicit delay control: directly tunes link latency changes.

Table 2, AdvNet.

Trace-Based Genetic Algorithm

Why not online RL?

- Requires fine-grained monitoring and live control of network conditions.
- Timing differences and execution noise make target/reference synchronization difficult.
- Stale or mismatched observations can make learning unstable.

Trace-Based Genetic Algorithm

Why not online RL?

- Requires fine-grained monitoring and live control of network conditions.
- Timing differences and execution noise make target/reference synchronization difficult.
- Stale or mismatched observations can make learning unstable.

Genetic algorithm (GA)

- Maintain a population of candidate traces T; each trace's numerical parameters are its genes.
- Fitness: the measured performance-gap score.
- Select promising parents; apply crossover and mutation to form the next generation.
- Repeat evaluation and selection until the budget expires.

GA Example: Crossover and Mutation

Two-point crossover

	Gene 1	Gene 2	Gene 3	Gene 4
Parent A	20	5	40	10
Parent B	60	30	15	80
Child A	20	30	15	10
Child B	60	5	40	80

e.g., Genes 1–2: bandwidth; Genes 3–4: one-way delay.

The vertical lines mark two cut points.

Swap genes 2 and 3; keep the outer genes from the original parent.

Aligned columns represent the same parameter, so crossover preserves bounds.

GA Example: Crossover and Mutation

Two-point crossover

	Gene 1	Gene 2	Gene 3	Gene 4
Parent A	20	5	40	10
Parent B	60	30	15	80
Child A	20	30	15	10
Child B	60	5	40	80

e.g., Genes 1–2: bandwidth; Genes 3–4: one-way delay.

The vertical lines mark two cut points.

Swap genes 2 and 3; keep the outer genes from the original parent.

Aligned columns represent the same parameter, so crossover preserves bounds.

Uniform integer mutation

Child A:

$[20, 30, 15, 10] \rightarrow [20, 30, 73, 10]$

With probability p, replace each gene by an integer sampled uniformly from its allowed range.

Comparing Optimization Algorithms

GA: genetic algorithm RG: random generation

BO: Bayesian optimization (random forest) EPS: ϵ-greedy search

Fig. 3a: one-hour budget

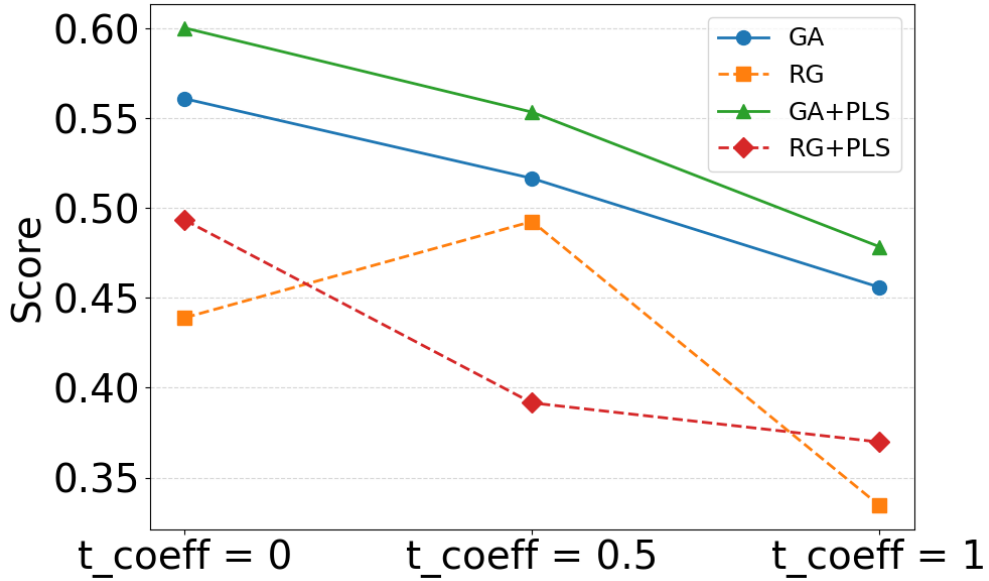


Fig. 3b: two-hour budget

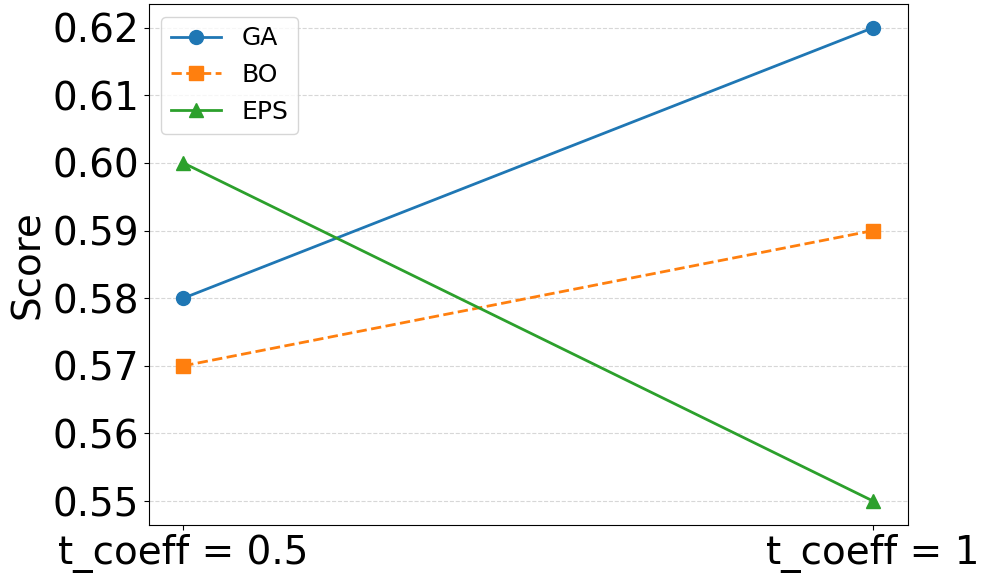


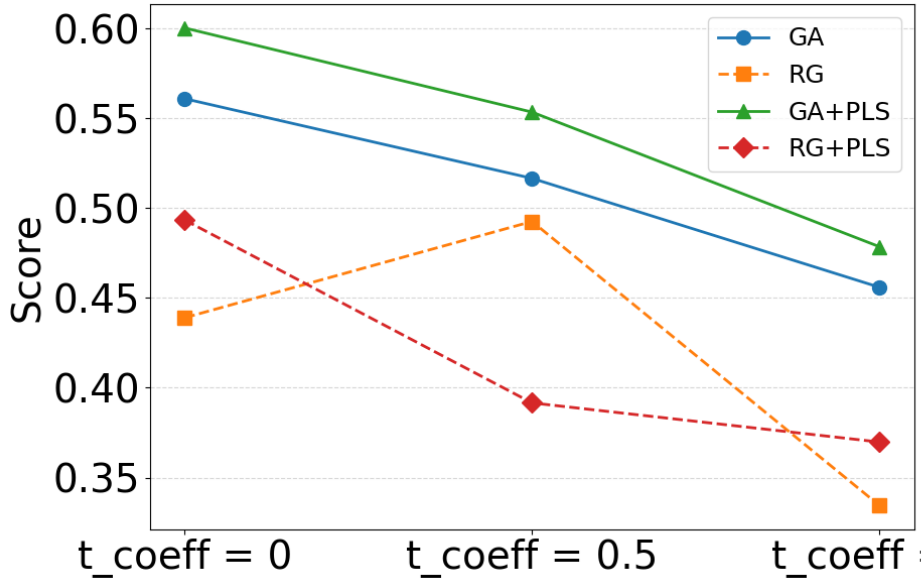
Fig. 3a–b, AdvNet.

Comparing Optimization Algorithms

GA: genetic algorithm RG: random generation

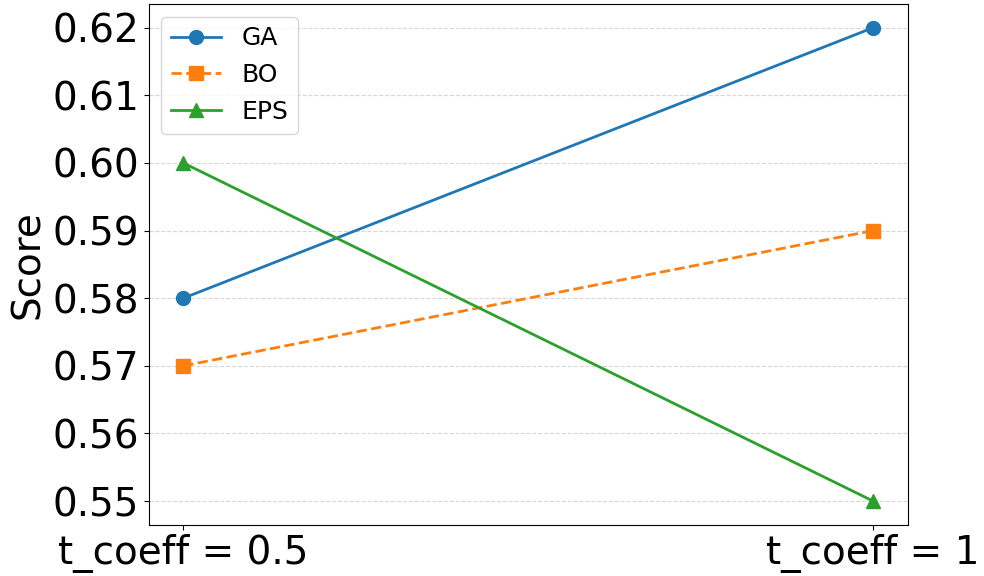
BO: Bayesian optimization (random forest) EPS: ϵ-greedy search

Fig. 3a: one-hour budget



GA beats RG across all three weights; execution feedback helps.

Fig. 3b: two-hour budget

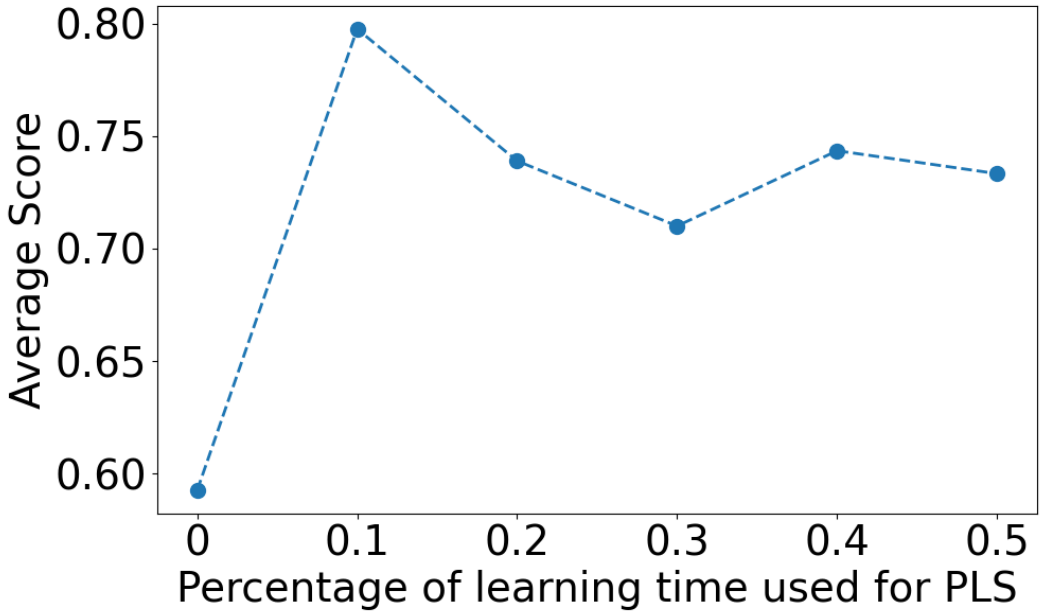


No clear winner across settings; GA has a slightly higher average.

Fig. 3a–b, AdvNet.

Post-Learning Selection and Noise Handling

SimpleMax selects the highest observed score; noise can inflate the winner.

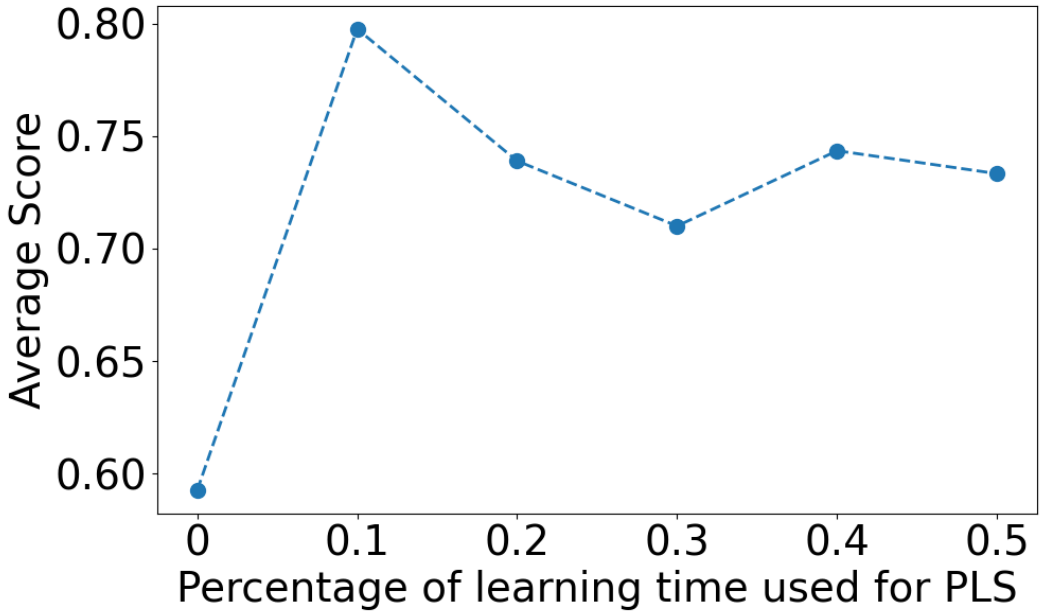


- During search: repeat each trace. Take median reference and target performance separately, then compute the score.
- PLS: reserve part of the budget to re-test promising traces and compare their mean scores.
- After selection: independently re-evaluate the chosen trace in a single instance.

Fig. 2, AdvNet.

Post-Learning Selection and Noise Handling

SimpleMax selects the highest observed score; noise can inflate the winner.



- During search: repeat each trace. Take median reference and target performance separately, then compute the score.
- PLS: reserve part of the budget to re-test promising traces and compare their mean scores.
- After selection: independently re-evaluate the chosen trace in a single instance.

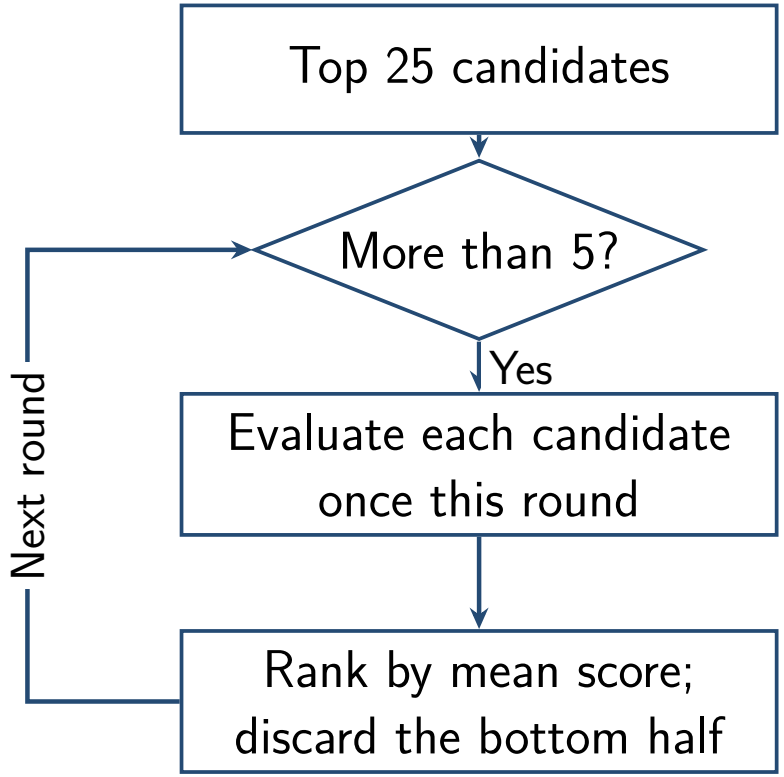
0% PLS: score $\approx 0.59 \quad \rightarrow \quad$ 10% PLS: score ≈ 0.80 (best).

Allocating more time to PLS leaves less time to explore new environments.

Fig. 2, AdvNet.

Multi-Round Elimination (MRE)

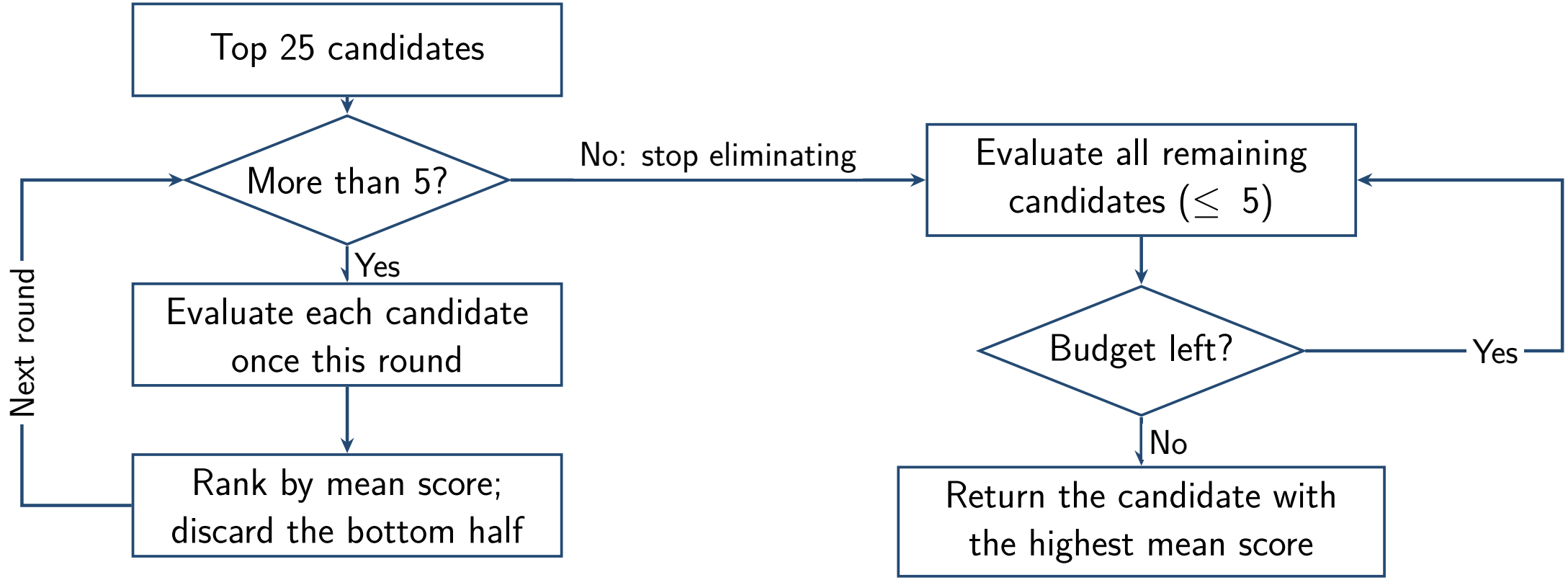
AdvNet uses MRE for its Post-Learning Selection (PLS) stage.



Mean scores use all evaluations collected so far during PLS.

Multi-Round Elimination (MRE)

AdvNet uses MRE for its Post-Learning Selection (PLS) stage.



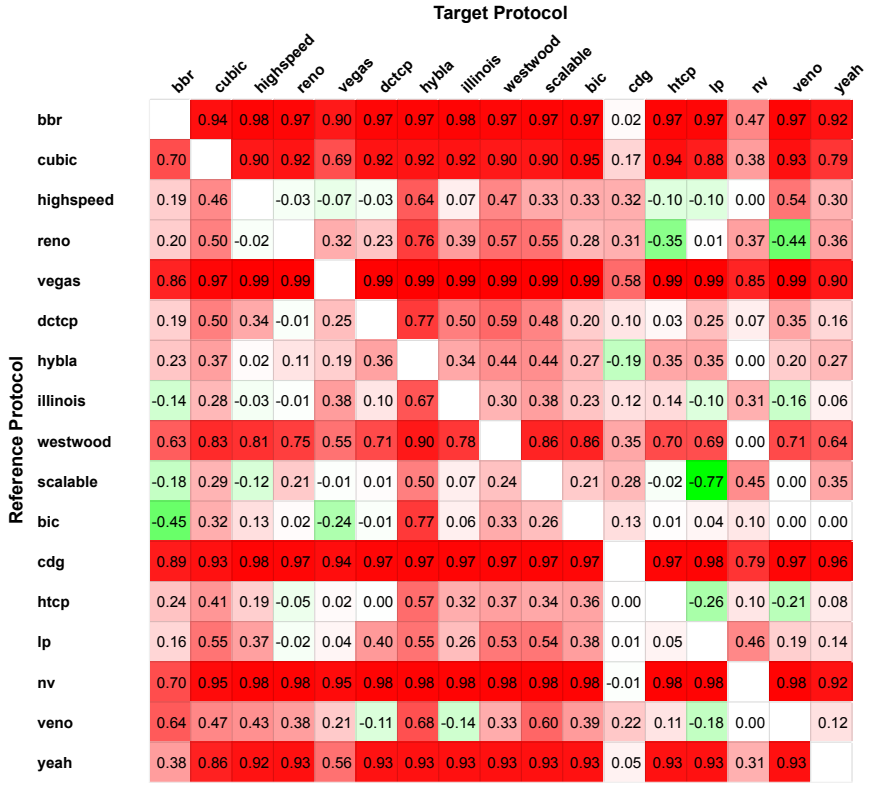
Mean scores use all evaluations collected so far during PLS.

Mapping Developer Tasks to Three Use Cases

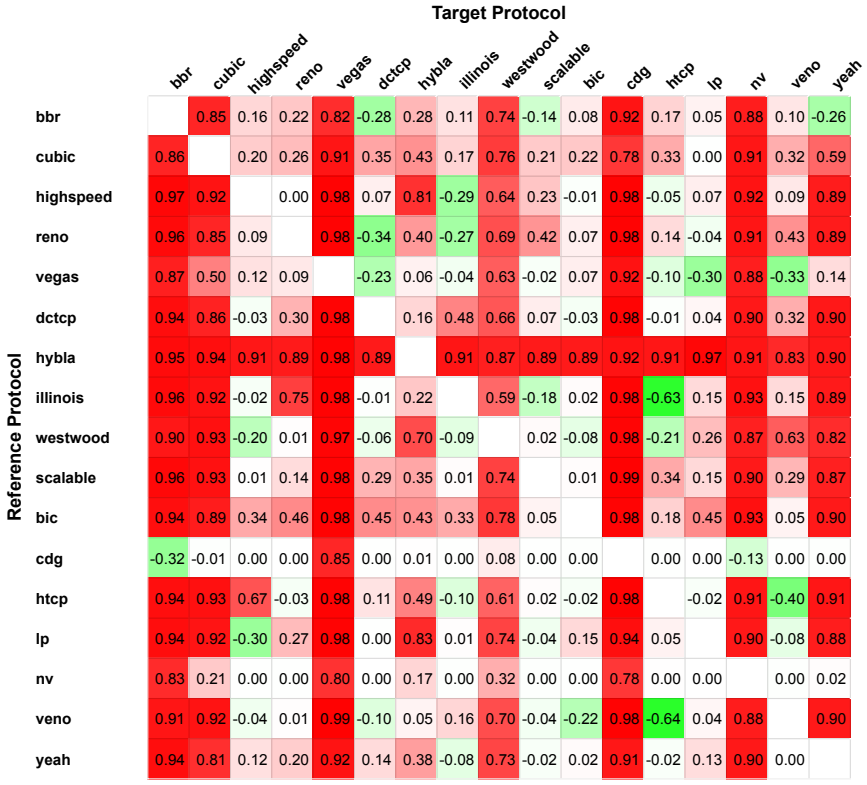
Use case	Reference (R)	Target (T)	What it reveals
UC1: Suboptimality	Capacity or another protocol	Protocol under test	Underutilization and implementation bugs
UC1: Fairness	A flow running alone	The same flow with a competing flow	Unfair bandwidth sharing and starvation
UC2: Regression testing	Older version, e.g., BBR v1	Newer version, e.g., BBR v3	Performance or fairness regressions
UC3: Design objectives	Single-path transport	Dual-path transport	Violations of the multipath "do no harm" goal

UC1: All-Pairs Comparison of 17 Kernel TCPs

$17 \times 16 = 272$ directed pairs $\times$ 2 objectives $(t_{\text{coeff}} = 0.5, 1) = 544$ scenarios.
Adversarial environments (score > 0) found in 448 of 544 scenarios.



(a) $t_{\text{coeff}} = 0.5$



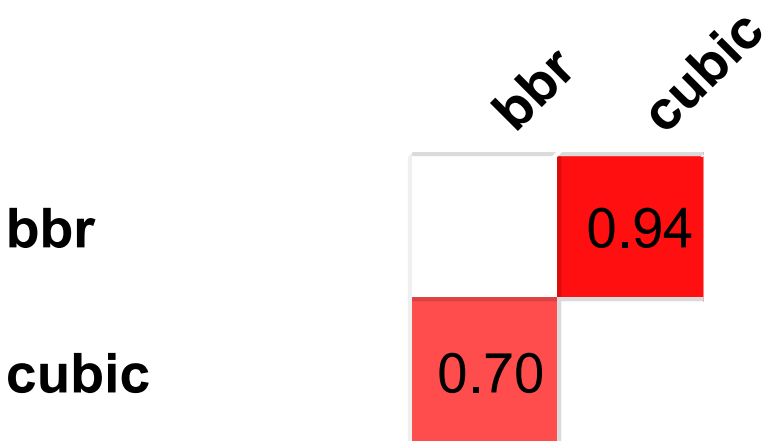
(b) $t_{\text{coeff}} = 1$

Fig. 4a–b, AdvNet.

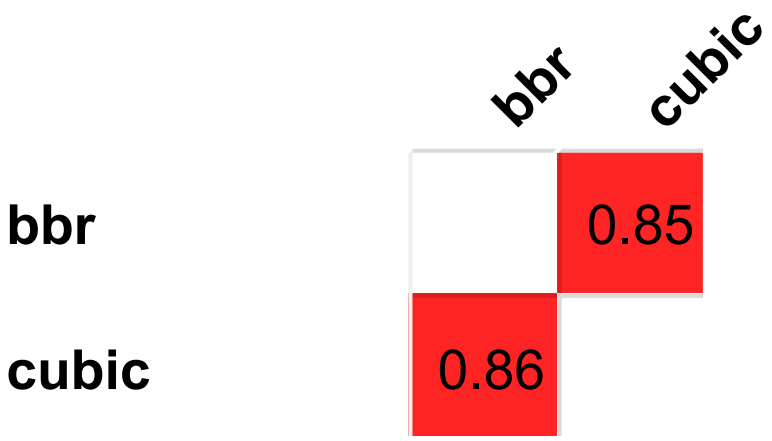
UC1: BBR and CUBIC Under Different Environments

Rows: reference protocol; columns: target protocol.

Equal throughput/delay weight ($t_{\text{coeff}} = 0.5$)



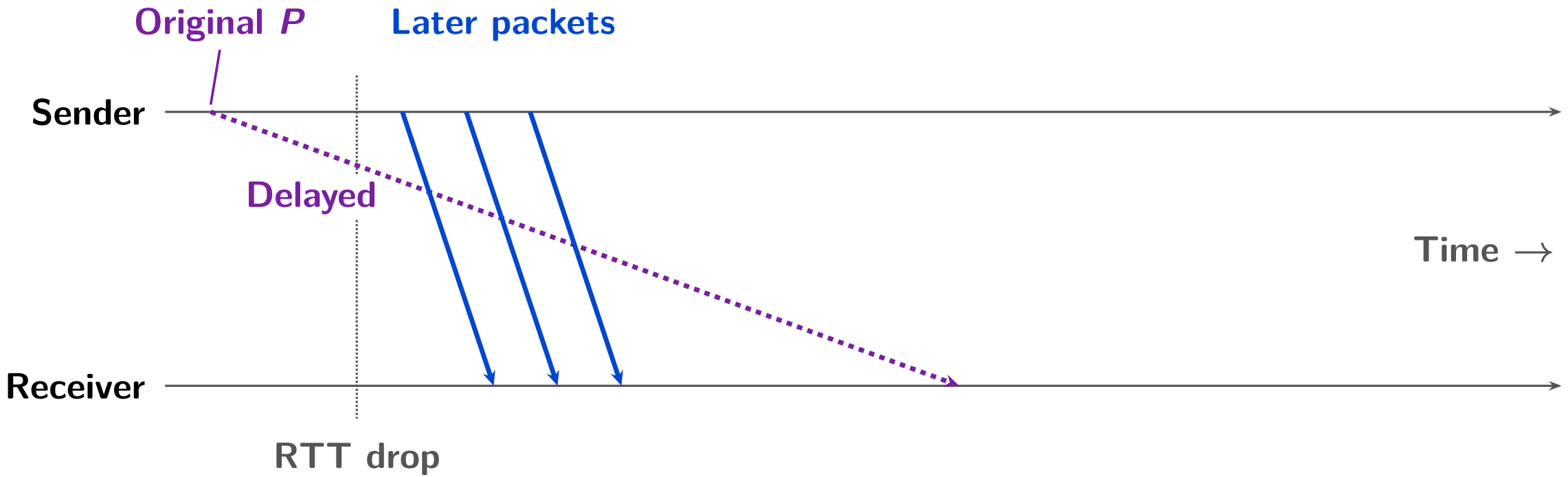
Throughput only ($t_{\text{coeff}} = 1$)



- Large performance gaps occur in both directions under both objectives.
- Each cell is a separate search under a different environment.

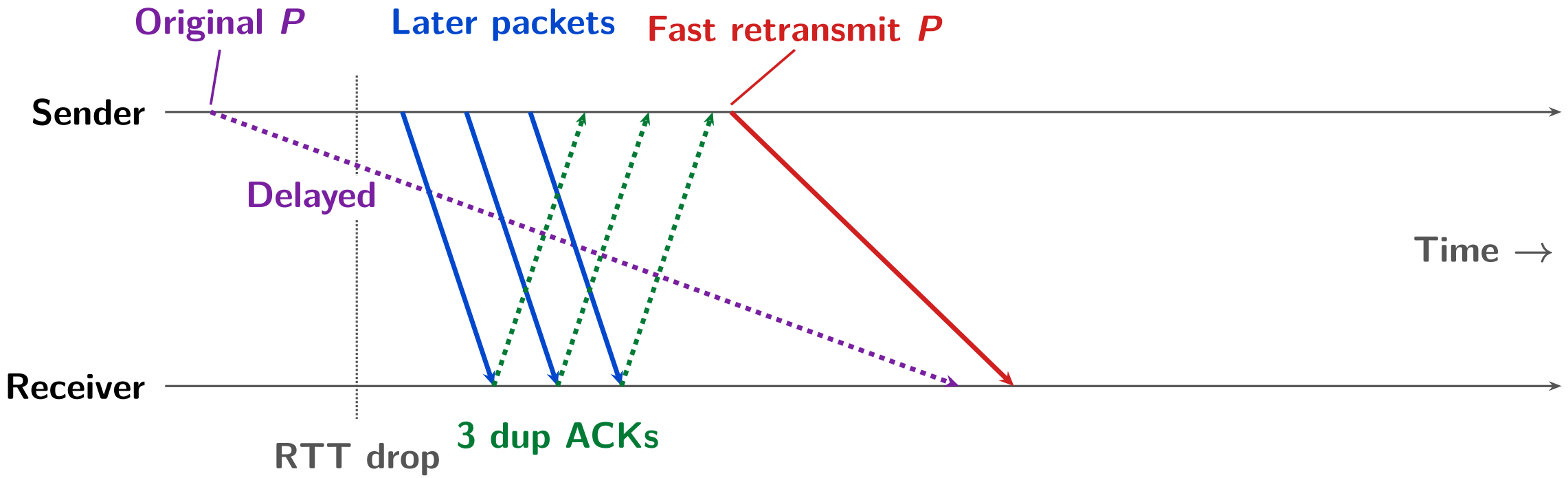
Fig. 4a–b (cropped), AdvNet.

UC1: Linux Kernel TCP ACK Bug



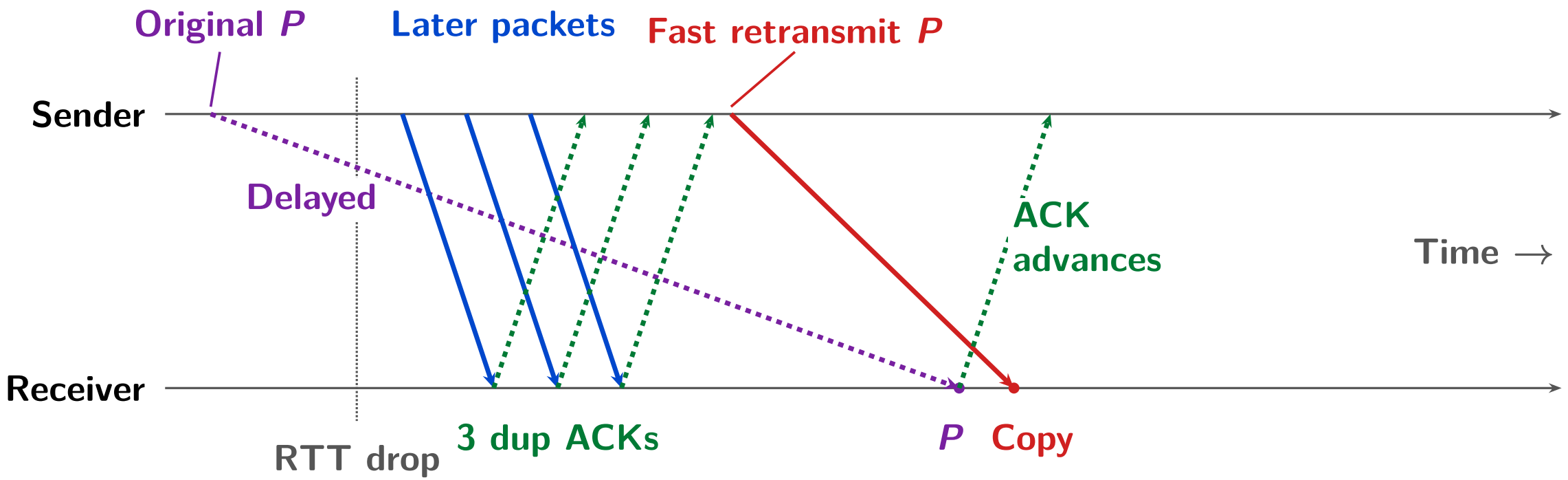
High RTT for ∼1 s, then a sharp drop: later packets overtake P.

UC1: Linux Kernel TCP ACK Bug



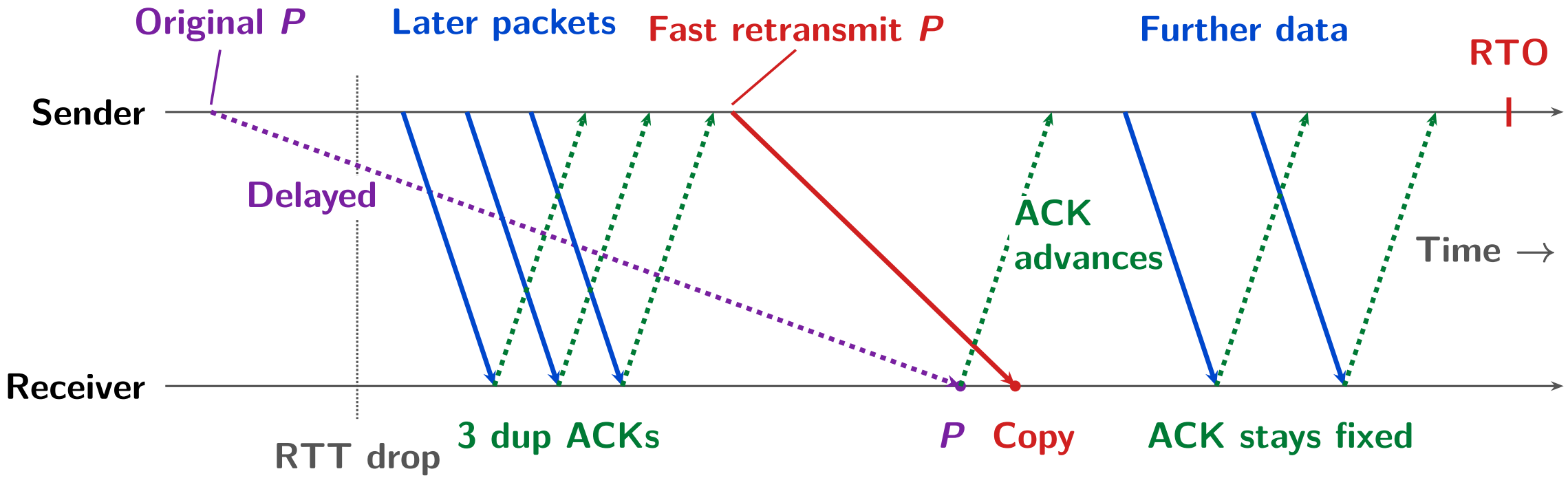
Three duplicate ACKs trigger fast retransmit while the original P is still in flight.

UC1: Linux Kernel TCP ACK Bug



The original P arrives first and advances the ACK number. Its retransmitted copy then triggers the ACK-update bug.

UC1: Linux Kernel TCP ACK Bug



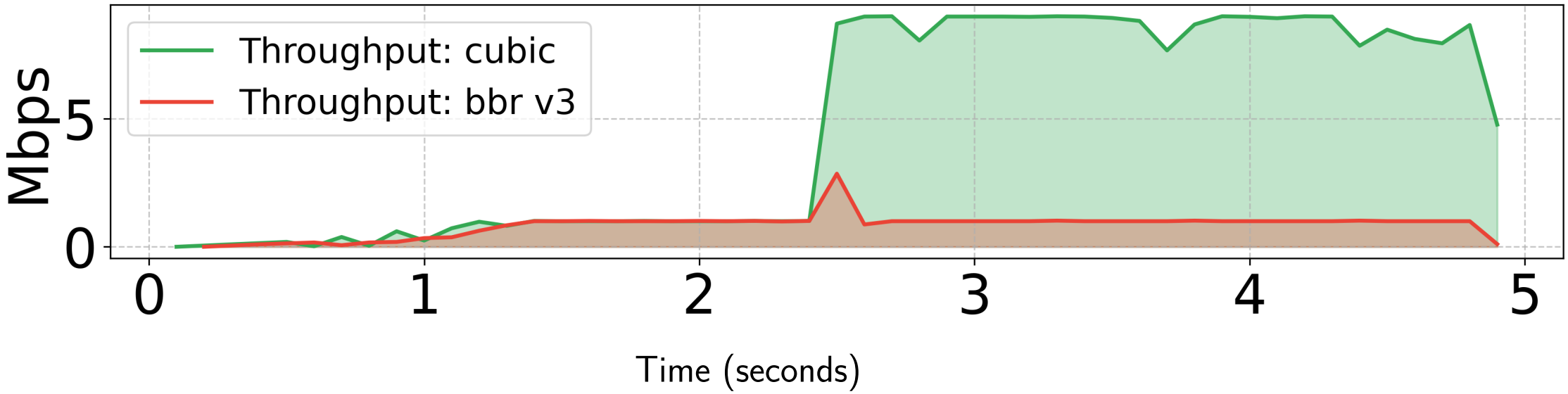
Further packets arrive, but the ACK number stays fixed; the sender eventually enters RTO.

Why BBR? Smooth pacing makes this arrival order easier to trigger.

Authors verified using eBPF and Wireshark.

UC1: BBR v3 Slow Convergence on a Newer Kernel

On kernel 6.13.7+, the earlier ACK bug was no longer observed.

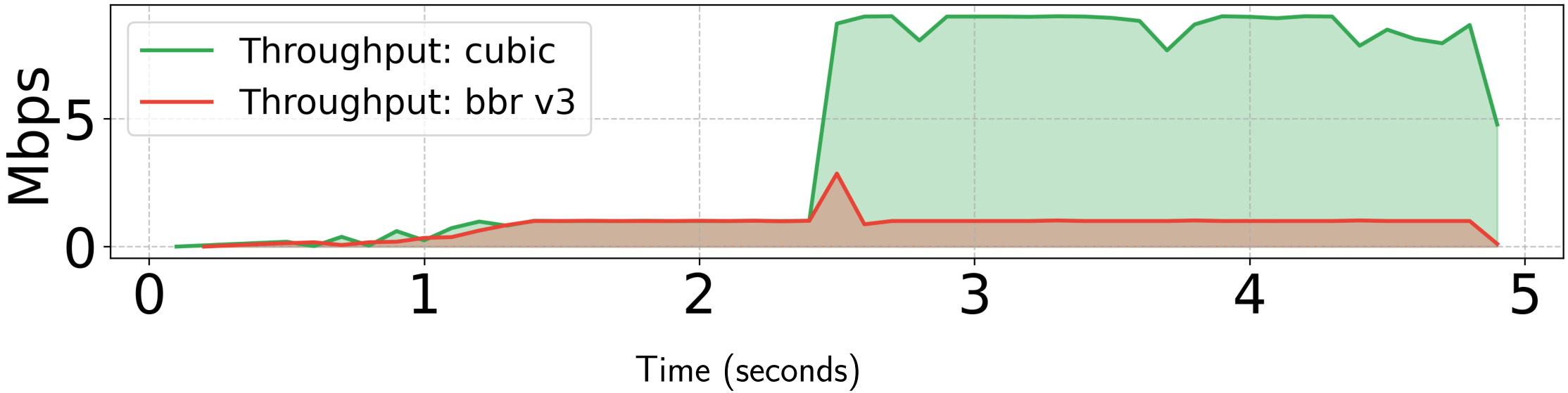


- CUBIC ramps up quickly; BBR v3 stays low over the 5-second run.

Fig. 6 (throughput panel), AdvNet.

UC1: BBR v3 Slow Convergence on a Newer Kernel

On kernel 6.13.7+, the earlier ACK bug was no longer observed.



- CUBIC ramps up quickly; BBR v3 stays low over the 5-second run.
- In a longer run, BBR v3 eventually ramps up: slow convergence, rather than a permanent stall.

Fig. 6 (throughput panel), AdvNet.

UC1: Fairness Between Competing Flows

Compare the same flow's throughput alone (R) and with a competitor (T).

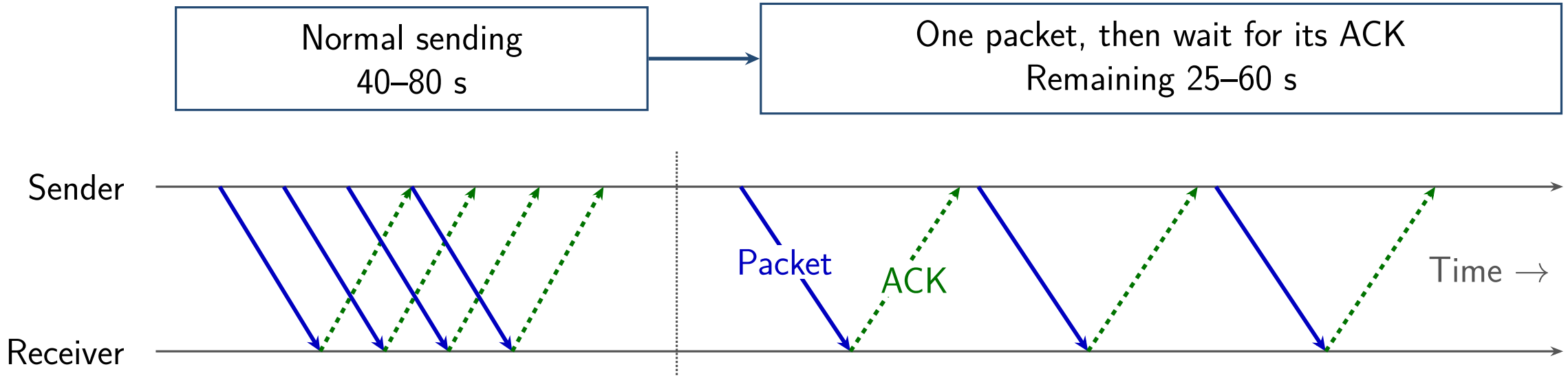
- CUBIC can take nearly all bandwidth and starve BBR.
- In other environments, BBR can starve CUBIC.

UC1: Fairness Between Competing Flows

Compare the same flow's throughput alone (R) and with a competitor (T).

- CUBIC can take nearly all bandwidth and starve BBR.
- In other environments, BBR can starve CUBIC.

Observed sending behavior

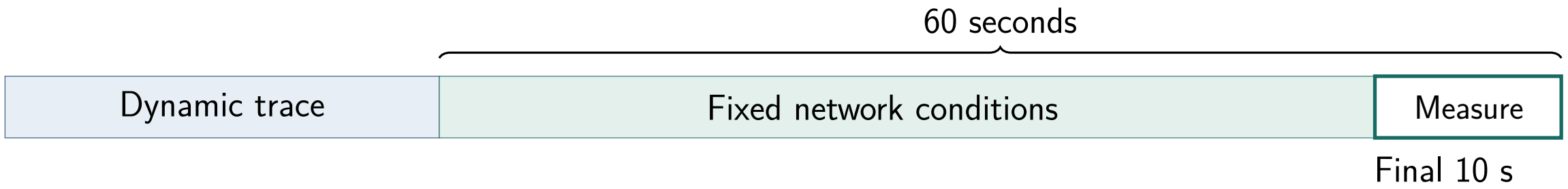


The BBR team confirmed that the reported environment was problematic.

UC2: Regression Testing Across BBR Versions

Fairness regression (same trace)

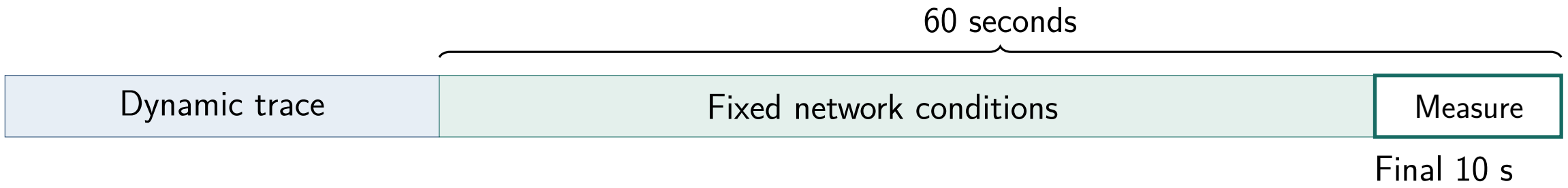
Reference (R): CUBIC throughput when competing with BBR v1.
Target (T): CUBIC throughput when competing with BBR v3.



UC2: Regression Testing Across BBR Versions

Fairness regression (same trace)

Reference (R): CUBIC throughput when competing with BBR v1.
Target (T): CUBIC throughput when competing with BBR v3.



- With BBR v1: CUBIC gets 12× BBR v1's throughput.
- With BBR v3: BBR v3 gets 11.6× CUBIC's throughput.

Separate single-flow test

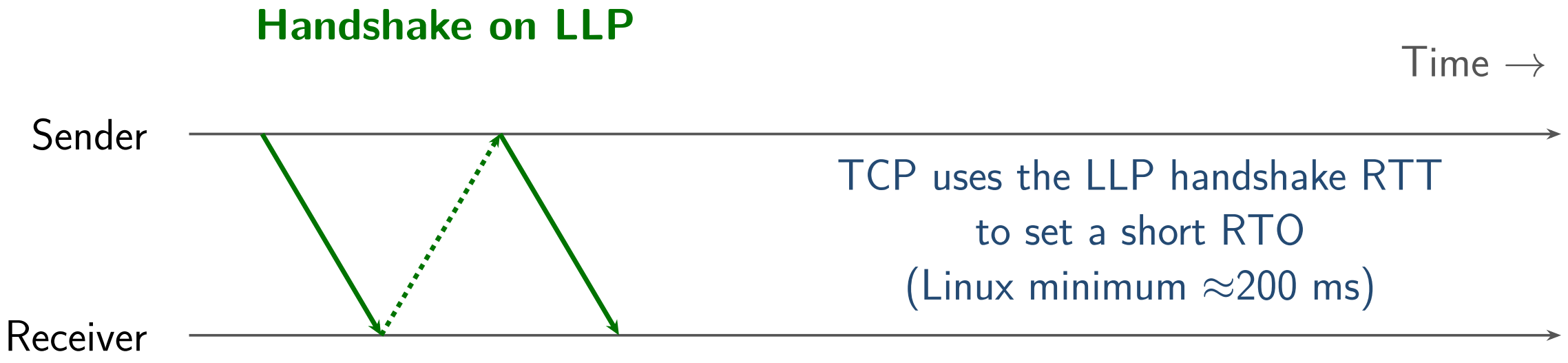
Without a competing CUBIC flow, AdvNet also finds traces where BBR v1 achieves more than 5× BBR v3's throughput.

UC3: DChannel Sets Up an RTT Mismatch

Discovered environment:

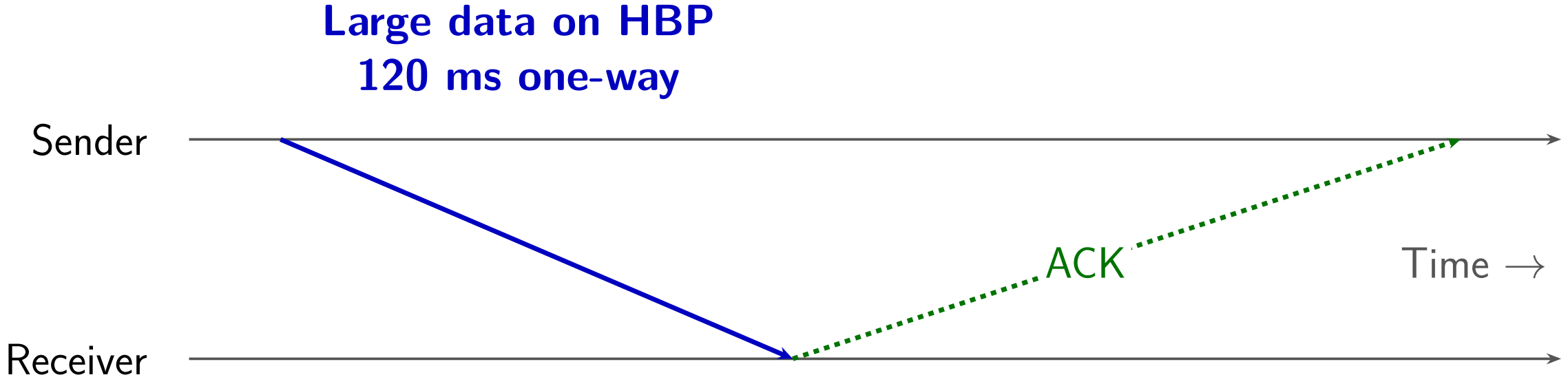
- HBP: high-bandwidth path, 120 ms one-way delay.
- LLP: low-latency path, 3 ms one-way delay.

Reference: HBP alone. Target: DChannel over both paths.



UC3: Slow-Path Data Outlasts the RTO

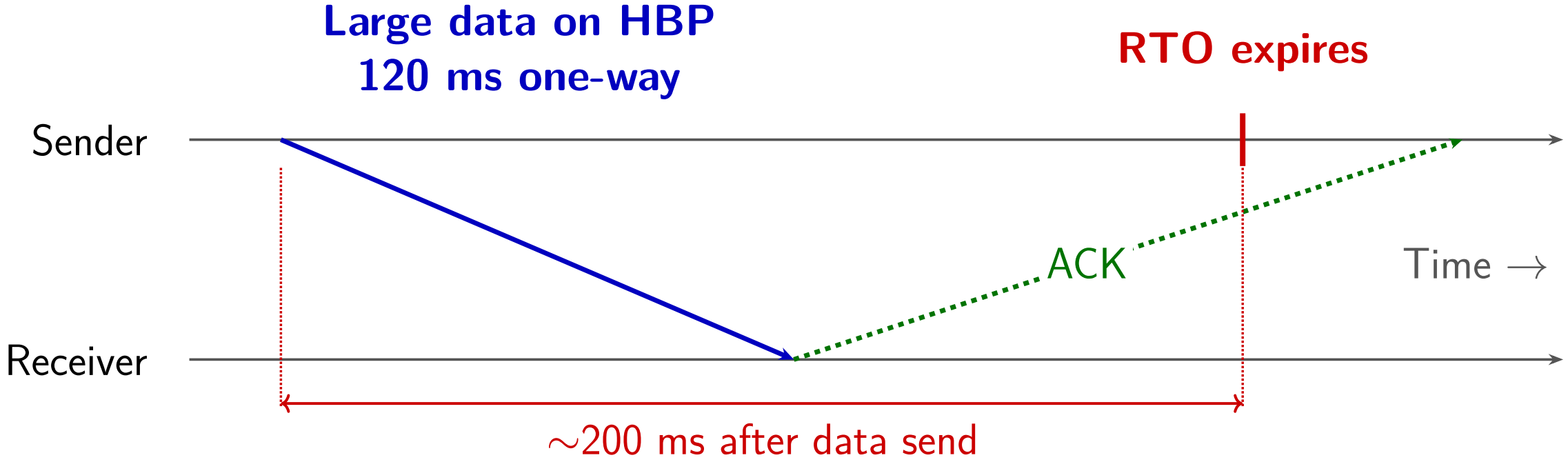
The RTO was set using the fast LLP handshake.



The data round-trip and queueing delay can exceed this short RTO.

UC3: Slow-Path Data Outlasts the RTO

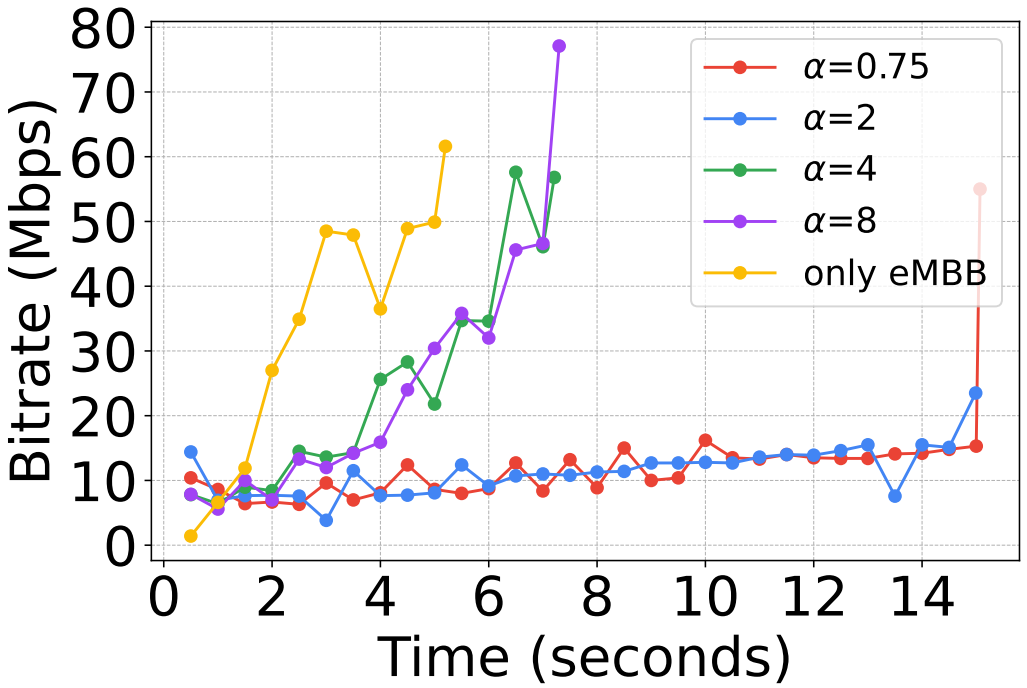
The RTO was set using the fast LLP handshake.



The RTO expires before the data ACK returns, causing spurious retransmission and a sending-rate reduction.

Throughput falls to about 1/3 of HBP alone (DChannel + CUBIC).

UC3: DChannel Evaluation Results



20 MB download

Earlier curve end means shorter flow completion time.

Packet-steering threshold

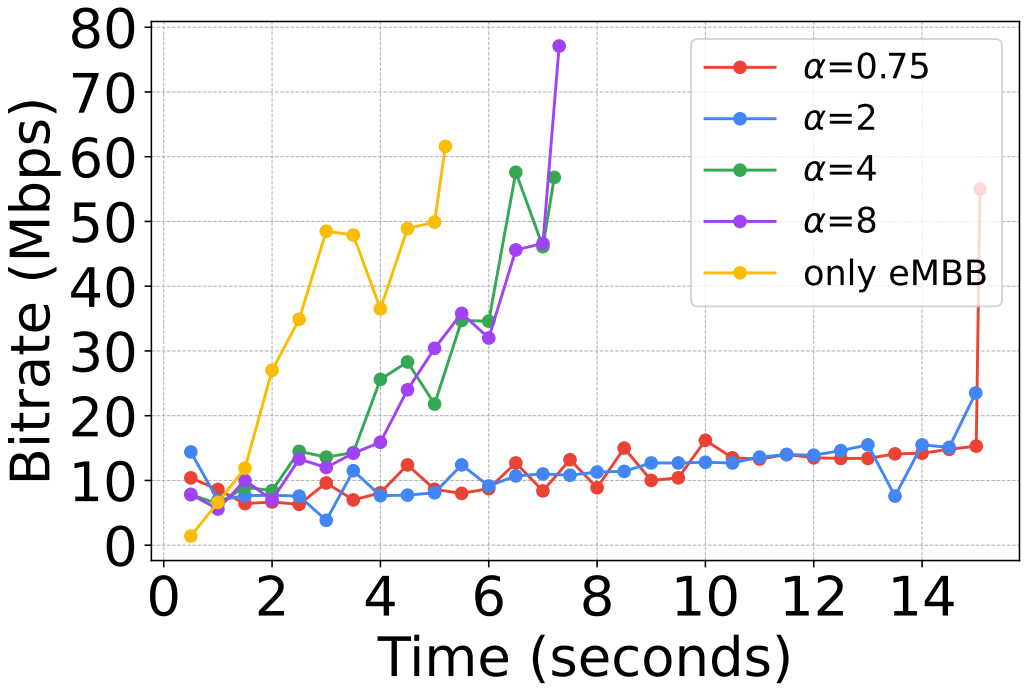
Send to LLP when reward $R > \alpha C$, where C is cost.

Default: $\alpha = 0.75$.

- All tested α values finish later than the single HBP ("only eMBB").

Fig. 7, AdvNet.

UC3: DChannel Evaluation Results



20 MB download

Earlier curve end means shorter flow completion time.

Packet-steering threshold

Send to LLP when reward $R > \alpha C$, where C is cost.

Default: $\alpha = 0.75$.

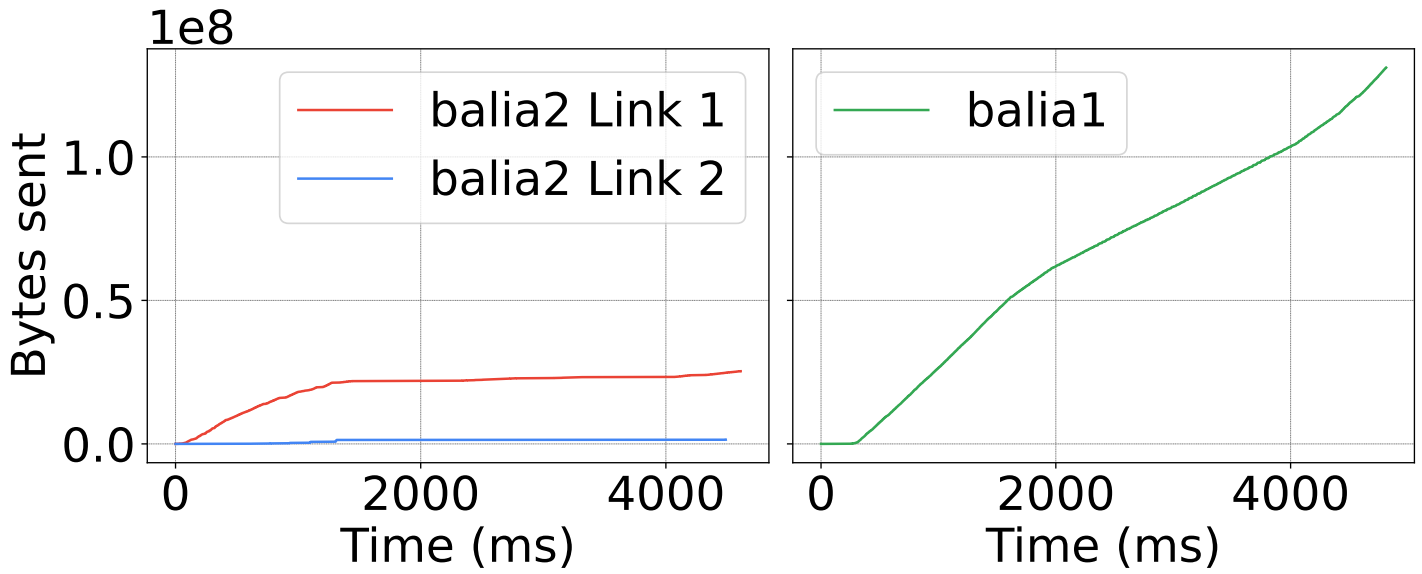
- All tested α values finish later than the single HBP ("only eMBB").
- Reducing HBP RTT below 200 ms restores DChannel's advantage over a single path, supporting the timeout diagnosis.

Fig. 7, AdvNet.

UC3: Single-Path vs. Dual-Path in Kernel MPTCP

AdvNet found two-path regressions in Linux Multipath TCP (MPTCP).

Allows one TCP connection to transmit over multiple paths.



Dual-path throughput is 85.5% lower in this environment.

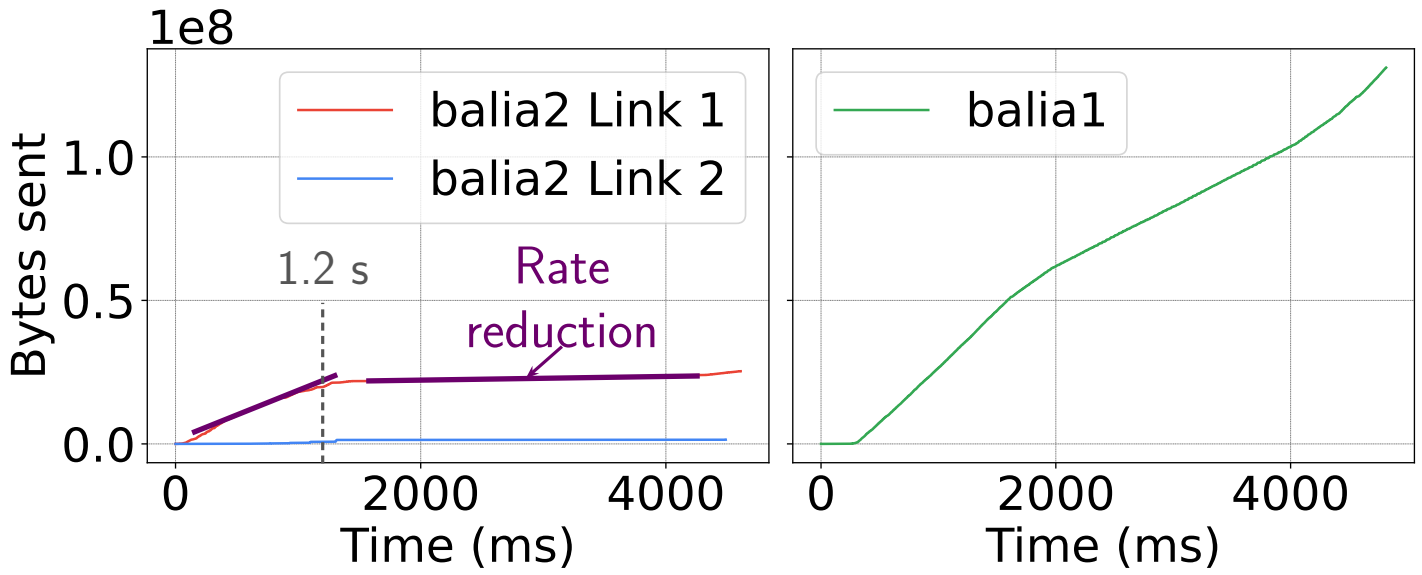
`balia2`: dual path; `balia1`: single path.

Fig. 8, AdvNet.

UC3: Single-Path vs. Dual-Path in Kernel MPTCP

AdvNet found two-path regressions in Linux Multipath TCP (MPTCP).

Allows one TCP connection to transmit over multiple paths.



Dual-path throughput is 85.5% lower in this environment.

Around 0.5 seconds

Single path overtakes dual path.

Around 1.2 seconds

A ∼40 ms ACK delay reduces both links' rates, despite available bandwidth on Link 1.

`balia2`: dual path; `balia1`: single path.

Fig. 8, AdvNet.

Conclusion and Takeaways

What AdvNet contributes

- Automated black-box search for performance failures in real kernel code.
- Custom references adapt the same pipeline to bugs, regressions, and design objectives.
- Compact representation and noise handling may matter more than optimizer choice: GA, BO, and EPS perform similarly.

Conclusion and Takeaways

What AdvNet contributes

- Automated black-box search for performance failures in real kernel code.
- Custom references adapt the same pipeline to bugs, regressions, and design objectives.
- Compact representation and noise handling may matter more than optimizer choice: GA, BO, and EPS perform similarly.

Scope and next steps

- Limited to bandwidth, delay, and queue size under simple workloads.
- Root-cause diagnosis remains manual; real-world frequency is not estimated.
- Simplify traces automatically and extend to adaptive bitrate video streaming.

Backup: Formal Verification and GANs

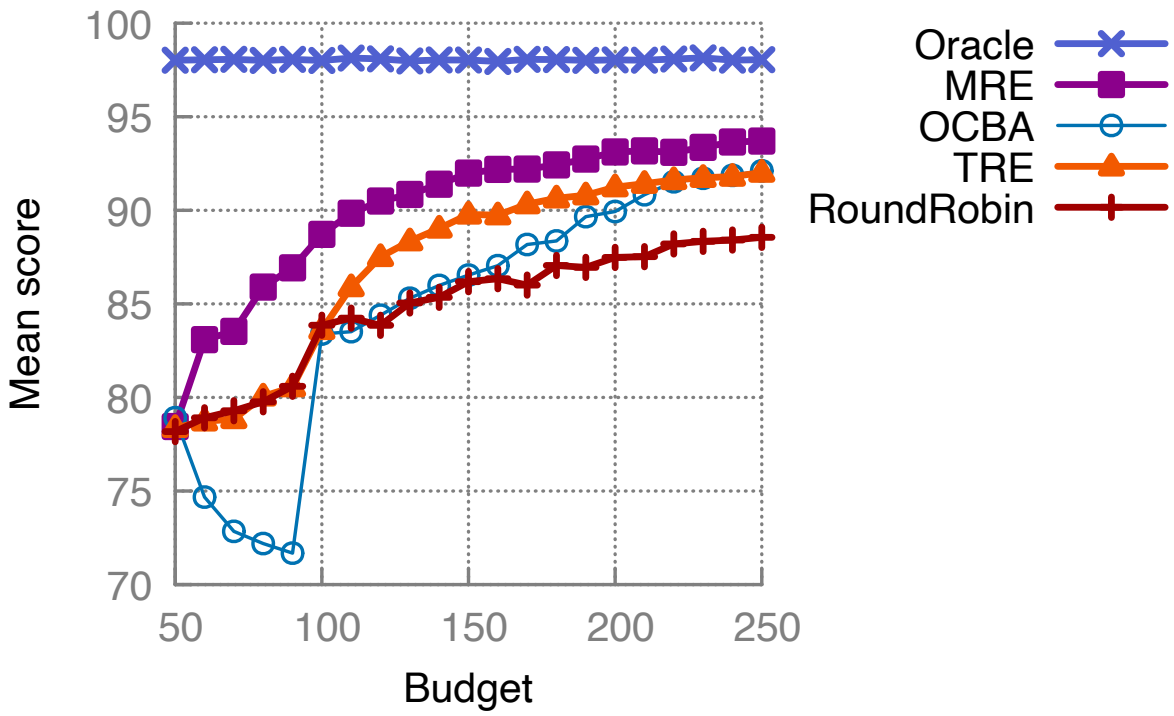
Formal verification

- Checks specified properties using model checking or symbolic analysis.
- Requires a precise model or specification; complex kernel code and changing networks create scaling challenges.
- AdvNet searches executions for counterexamples; it does not prove their absence.

Generative adversarial networks

- Typically learn a data distribution from existing examples.
- A dataset of challenging environments is not already available here.
- AdvNet uses measured target/reference performance gaps as search feedback.

Backup: Alternative PLS Methods



50 Gaussian alternatives; true means uniform on $[0, 100]$.

True mean of the selected alternative, averaged over 2,000 trials.

- RoundRobin: equal budget per candidate; return highest mean.
- OCBA: more runs for uncertain candidates and close contenders.
- MRE: discard weaker candidates in rounds, then select by mean.

MRE performs best; OCBA struggles below 100 samples.

OCBA: Optimal Computing Budget Allocation.

Fig. 11, AdvNet.

Backup: GA Operators in Equations

Parents a, b; gene index i; cut points $k_1 < k_2$; bounds $[l_i, u_i]$.

Two-point crossover

$$c_i = \begin{cases} b_i & k_1 < i \le k_2, \\ a_i & \text{otherwise,} \end{cases} \qquad d_i = \begin{cases} a_i & k_1 < i \le k_2, \\ b_i & \text{otherwise.} \end{cases}$$

Aligned coordinates keep each parameter within its own bounds.

Backup: GA Operators in Equations

Parents a, b; gene index i; cut points $k_1 < k_2$; bounds $[l_i, u_i]$.

Two-point crossover

$$c_i = \begin{cases} b_i & k_1 < i \le k_2, \\ a_i & \text{otherwise}, \end{cases} \qquad d_i = \begin{cases} a_i & k_1 < i \le k_2, \\ b_i & \text{otherwise}. \end{cases}$$

Aligned coordinates keep each parameter within its own bounds.

Uniform integer mutation

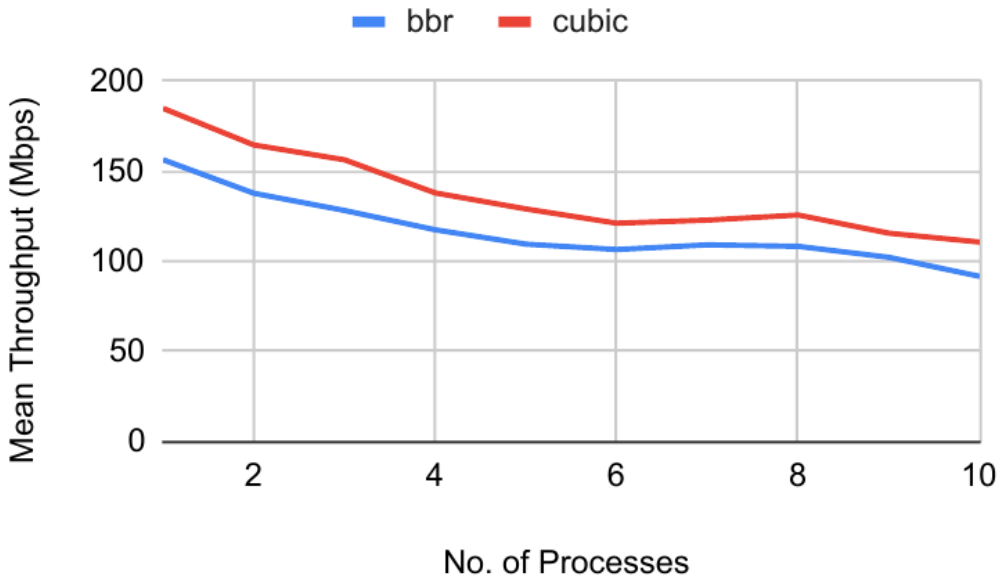
$$m_i = \begin{cases} U_i & \text{with probability } p, \\ c_i & \text{with probability } 1 - p, \end{cases} \qquad U_i \sim \text{UniformInteger}(l_i, u_i).$$

Each selected gene is resampled as an integer accepted by Mahimahi.

Backup: Parallel Execution and Measurement Interference

Each instance pins 3 CPU cores: emulator, `iperf` client, and `iperf` server.

Controlled test: 1 Gbps bandwidth, 5 ms one-way delay, 30-core server.



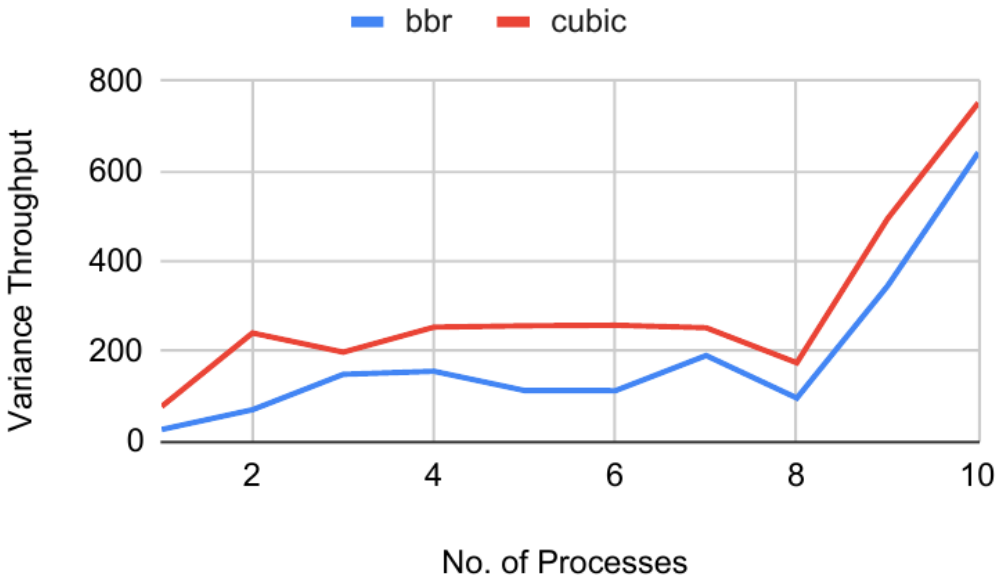
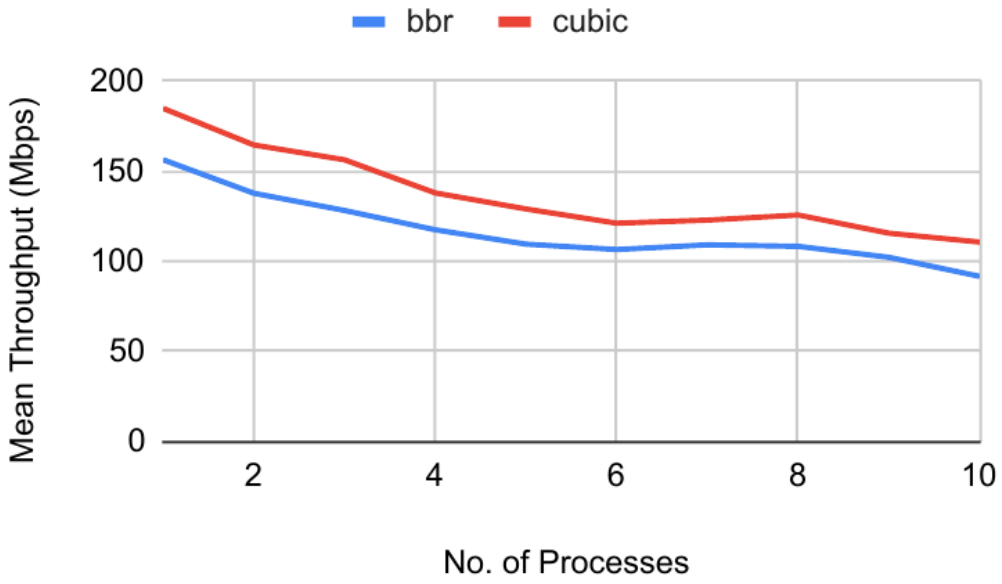


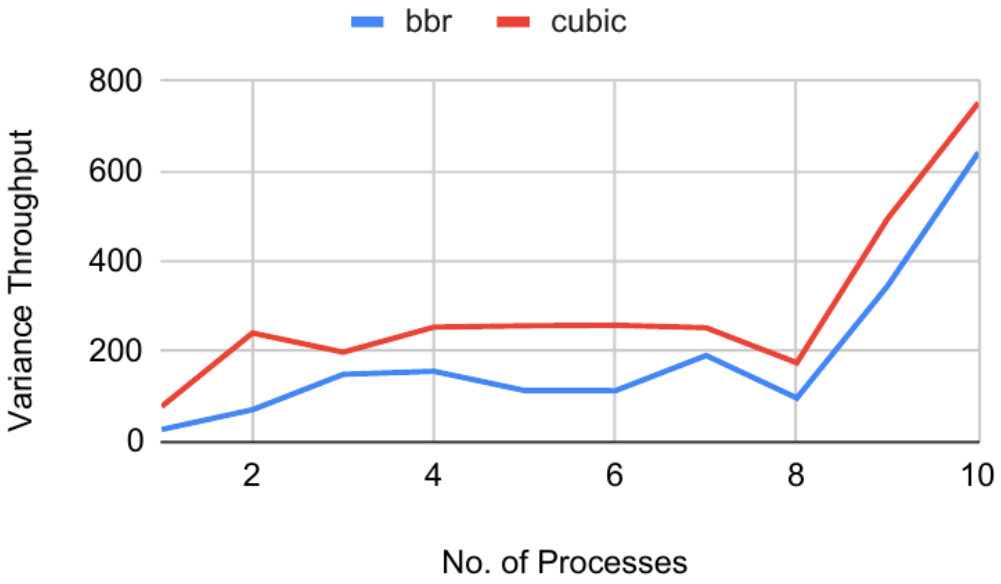
Fig. 9 (throughput panels), AdvNet.

Backup: Parallel Execution and Measurement Interference

Each instance pins 3 CPU cores: emulator, `iperf` client, and `iperf` server.

Controlled test: 1 Gbps bandwidth, 5 ms one-way delay, 30-core server.

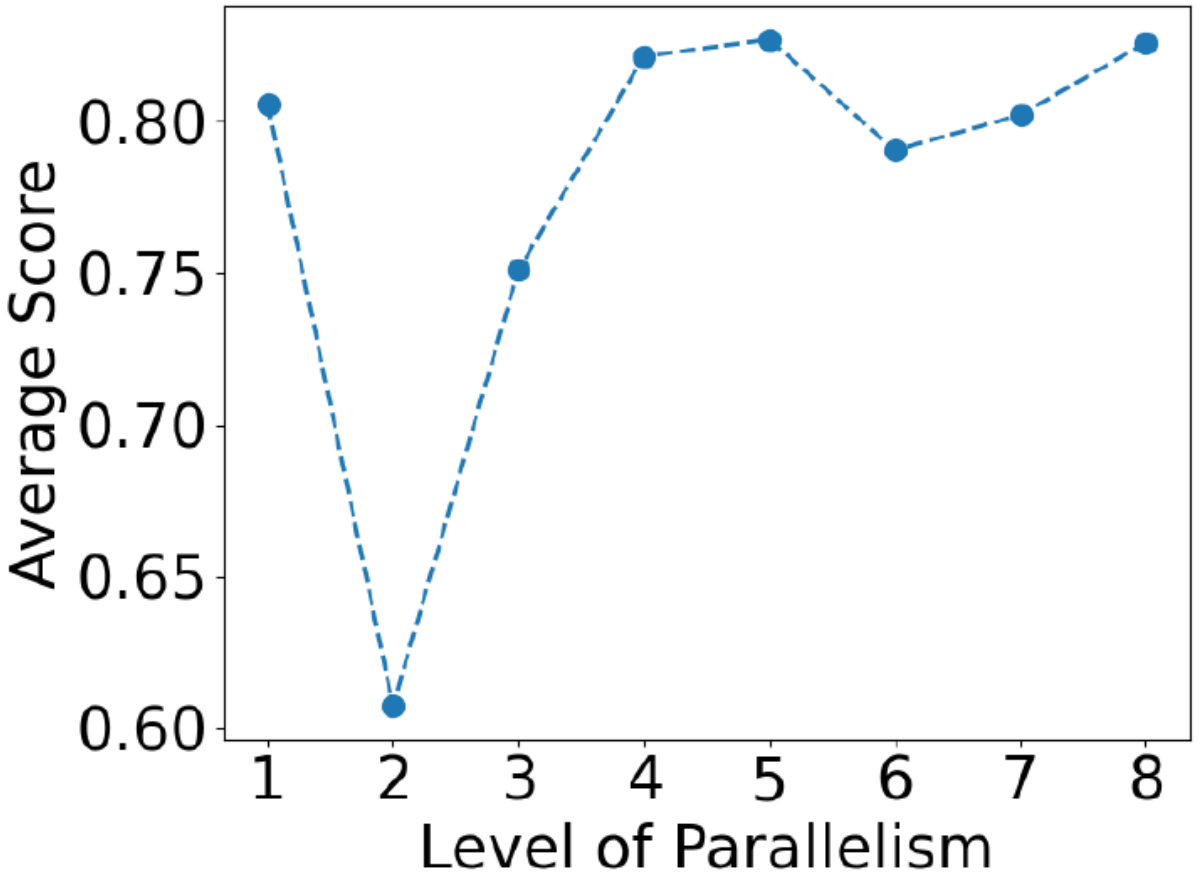




- Mean throughput falls as contention grows; variance rises sharply at 9–10 runs.
- The authors cap parallelism at 8 to avoid the high-variance region.

Fig. 9 (throughput panels), AdvNet.

Backup: Choosing the Level of Parallelism



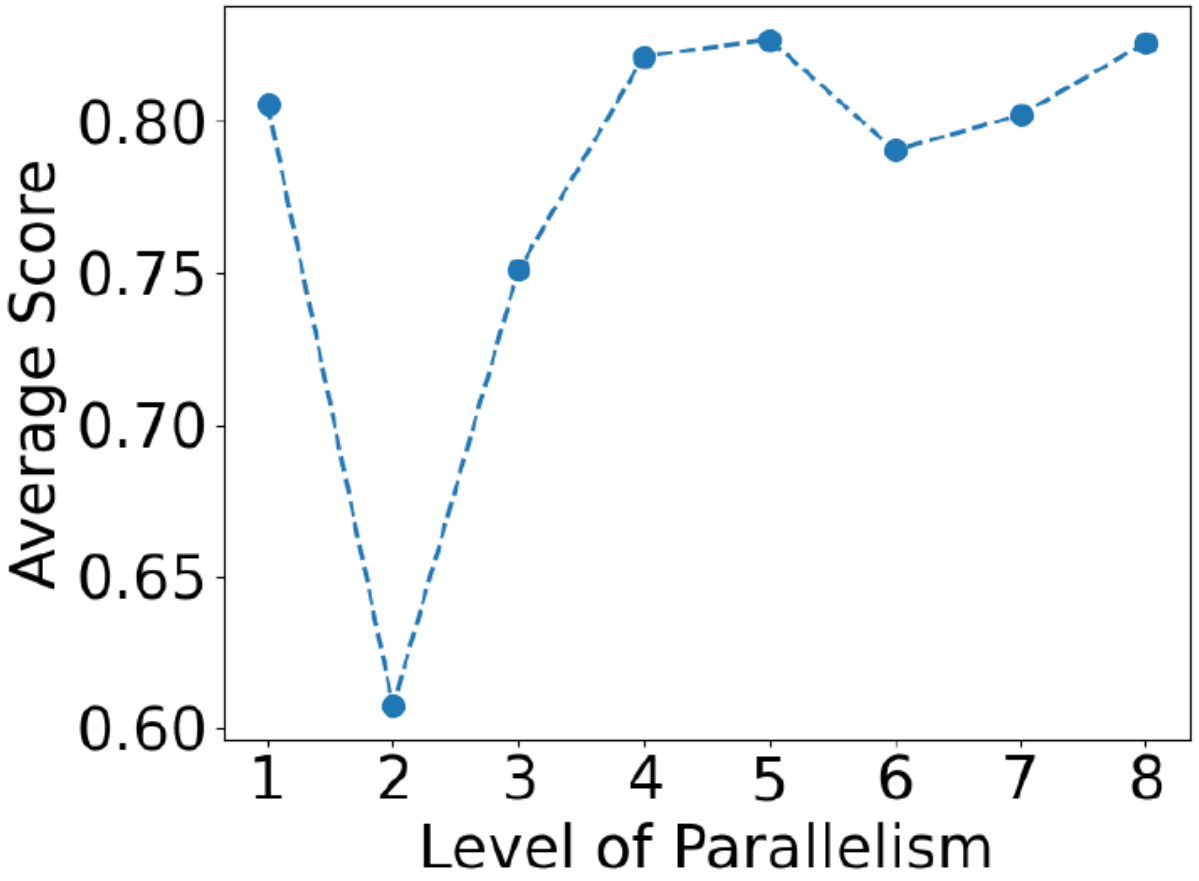
250 GA iterations; average across four target protocols.

Parallel runs evaluate the same candidate environment.

- From 1 to 2 runs, contention distorts throughput and lowers the final score.

Fig. 10, AdvNet.

Backup: Choosing the Level of Parallelism



250 GA iterations; average across four target protocols.

Parallel runs evaluate the same candidate environment.

- From 1 to 2 runs, contention distorts throughput and lowers the final score.
- More samples reduce noise; 5-way parallelism gives the best result.
- Improvement over 1 run is small; a single run saves CPU resources.

Final output scores are always evaluated using a single isolated instance.

Fig. 10, AdvNet.